UNIVERSITÀ
DI TORINO

Università degli Studi di Torino

Corso di Laurea Magistrale in Fisica dei Sistemi Complessi

Improving VAE Representational Capabilities for Understanding LLMs

Tesi di Laurea Magistrale

Relatore:
Prof. Michele Caselle

Correlatori:
Prof. Alan Perotti
Prof. André Panisson

Candidato:
Francesco Marchisotti

Tutor Aziendale:
Gabriele Ciravegna

Anno Accademico 2025/2026

[26th June 2026 at 14:46 – classicthesis v4.8]

Francesco Marchisotti: *Improving VAE Representational Capabilities for Understanding LLMs,* Fisica dei Sistemi Complessi

CENTAI TEAM:
Gabriele Ciravegna
Alan Perotti
André Panisson
Francesco Cozzi

UNITO PROFESSORS:
Michele Caselle
Matteo Osella

LOCATION:
Torino

TIME FRAME:
Oct. 2025 - Jul. 2026

"I wish there was a way to know you're in the good old days
before you've actually left them."

— *Andy Bernard*

ABSTRACT

Concept-based methods represent a model's computation in terms of human-meaningful concepts, but they typically encode each concept as a single scalar coefficient, e. g., a unit in the bottleneck of a Concept Bottleneck Model, or an atom in the dictionary of a sparse autoencoder (SAE). This scalar encoding trades representational capacity for a compact, one-number-per-concept code: each concept can vary only in magnitude, so reconstructing the underlying representation faithfully tends to require many concepts active at once. This thesis introduces the Variational Auto Embedding Encoder (VAEE), a model derived from first principles that represents each concept as a learned vector embedding instead, drawing together ideas from Concept Embedding Models, variational autoencoders, and vector-quantised variational autoencoders. The encoder maps an input to one vector embedding per concept; a learned prototype for each concept then decides, by similarity, which concepts are active; finally, the input is reconstructed from the stack of active embeddings. Its capabilities are demonstrated on one application, the decomposition of large language model activations, where the VAEE matches the reconstruction of a parameter-matched TopK SAE using roughly an order of magnitude fewer active concepts — evidence that vector-valued concepts carry more information per unit of sparsity. The thesis is deliberate about the limits of this evidence: reconstruction was assessed only by mean squared error, each concept now spans several active neurons rather than one, and the recovered concepts were not found more interpretable than the SAE's. The VAEE's contribution, then, lies in this added representational capacity; the interpretability of its concepts, their use for steering, and its application to representation learning more broadly are left for future work.

CONTENTS

LIST OF FIGURES

LIST OF TABLES

ACRONYMS

C-XAI Concept-based eXplainable Artificial Intelligence . . . 7, 19

CB-LLM Concept Bottleneck Large Language Model 9, 33

CBM Concept Bottleneck Model . . . . . v, 1, 8, 18 f., 47

CE cross-entropy 3, 17, 35, 48

CEM Concept Embedding Model . . . . . v, 1, 8, 18 ff., 47

ELBO evidence lower bound 12, 23, 28, 47

GELU Gaussian error linear unit 36, 43

KL Kullback–Leibler . . . . . 12 f., 27–30, 36, 38 f., 49

LCBM Learnable Concept-based Model 20

LF-CBM Label-free Concept Bottleneck Model . . . . . 8 f.

LLM large language model . . v, 1–4, 6, 8 ff., 17, 19, 31, 33, 44, 47 f.

MechInt Mechanistic Interpretability 1, 10, 14, 16, 34

MLP multi-layer perceptron . . . . . 4 f., 16 f., 43 f.

MSE mean squared error . . . . . v, 10, 12, 28, 35, 37, 43, 47 f.

SAE sparse autoencoderv, 1 f., 10, 13–20, 30 f., 33–38, 40 ff., 44 f., 47 f., 51

SST-2 Stanford Sentiment Treebank 33, 45

VAE variational autoencoder . . . . . v, 1 f., 10–13, 18, 20, 26, 47 f.

VAEE Variational Auto Embedding Encoder . v, 1 f., 19–22, 26, 30 f., 33–38, 40–44, 47 f., 51

VQ-VAE vector-quantised variational autoencoder v, 1 f., 13, 18, 20, 47 f.

1

INTRODUCTION

1.1 MOTIVATION

Large language models (LLMs) are remarkably capable, yet how they arrive at their outputs remains largely opaque: their behaviour emerges from billions of parameters and high-dimensional activations that resist direct inspection. Mechanistic Interpretability sets out to make this computation legible by reading it off the network's internal activations rather than treating the model as a black box [9, 28]. A central difficulty is that these activations are dense and polysemantic — a single direction in activation space takes part in many unrelated computations — so the first task is to decompose them into units a person can reason about.

Concept-based methods take up this task by describing a model's computation in terms of human-meaningful *concepts*. In a Concept Bottleneck Model the network is forced through a bottleneck whose units stand for named concepts [22]; in a sparse autoencoder (SAE) an overcomplete dictionary is learned so that each activation becomes a sparse combination of concept atoms [4, 41]. What these methods share is also one of their limitations: each concept is encoded as a single scalar coefficient and can vary only in magnitude. A scalar carries little information, so reconstructing an activation faithfully tends to require many concepts active at once — the sparsity–reconstruction trade-off that runs through the SAE literature, where a faithful reconstruction needs many active concepts but an interpretable decomposition requires only a few.

This thesis introduces the Variational Auto Embedding Encoder (VAEE), a model that lifts this restriction by representing each concept as a learned vector embedding rather than a scalar. The encoder maps an input to one embedding per concept; a learned prototype for each concept decides, by similarity, which concepts are active; and the input is reconstructed from the stack of the active embeddings. Assigning a concept to a vector rather than a scalar restores the representational capacity a scalar bottleneck gives up — the move that Concept Embedding Models make in the supervised setting [11] — and the VAEE carries it into the unsupervised decomposition of activations, combining it with the variational inference of variational autoencoders (VAEs) and the discrete latents of vector-quantised variational autoencoders (VQ-VAEs). The model is derived from first principles rather than assembled by analogy, and that derivation is the contribution of this thesis.

The architecture is demonstrated on one application, the decomposition of LLM activations, where it is compared against a TopK SAE on the same representations. Whether the concepts it recovers are interpretable, whether they can be used to steer model behaviour, and how far it carries over to representation learning are not settled here, and are left as directions for future work rather than claims this thesis makes.

1.2 THESIS STRUCTURE

The remainder of this thesis is organised as follows. Chapter 2 assembles the background the contribution builds on: the transformer and the residual-stream view of LLMs, the development of explainability from attribution to concepts, and the autoencoder family from VAEs and VQ-VAEs to SAEs. Chapter 3 develops the VAEE, deriving it from first principles as a probabilistic generative model and presenting its training objective. Chapter 4 evaluates the model on the decomposition of LLM activations against a parameter-matched TopK SAE, reporting the sparsity–reconstruction frontier, the recovered concept dictionaries, and a set of ablations. Chapter 5 draws together the contributions and lays out the open directions.

2

BACKGROUND AND RELATED WORK

This chapter establishes the background on which the thesis builds and reviews the related work, moving from the setting in which the contribution is demonstrated to the ideas from which it is built. Section 2.1 introduces large language models and the transformer architecture that produces the dense internal activations on which that demonstration rests. Section 2.2 reviews explainable AI, tracing the field's shift from attributing a prediction to input features towards explaining it in terms of human-meaningful *concepts*, the view this work adopts. Section 2.3 then develops the autoencoder family, from the standard autoencoder through the variational and sparse variants between which the thesis's contribution sits.

2.1 NATURAL LANGUAGE PROCESSING

Natural Language Processing is the branch of machine learning concerned with getting computers to process and generate human language. For most of its history the field advanced through task-specific systems — separate models for translation, sentiment analysis, or question answering — but it has since converged on a single recipe: train one large neural network to predict text on a vast corpus, then reuse it across tasks. The large language models (LLMs) that result are the systems on which the thesis's contribution is demonstrated. What follows introduces only as much of their training and architecture as the later chapters rely on, with the emphasis on the internal activations these models compute rather than the text they emit.

2.1.1 LANGUAGE MODELLING

A language model defines a probability distribution over sequences of text. In the dominant *autoregressive* formulation, this distribution is factorised left to right, so that the model repeatedly predicts the next unit of text given all the units before it [30]. The units are *tokens*, words or sub-word fragments drawn from a fixed vocabulary, and the model is a neural network that maps a sequence of tokens to a probability distribution over the vocabulary for the next position. Training minimises the cross-entropy (CE) between this predicted distribution and the true next token, averaged over a large corpus; the objective requires no human annotation, since the supervision signal is the text itself.

Scaling this recipe to more parameters, more data, and more computing power yields LLMs whose capabilities extend well beyond the literal next-token task they were trained on. A network trained only to continue text acquires the ability to translate, summarise, answer questions, and follow instructions, often from nothing more than a description and a few examples supplied in its input, with no change to its weights [5, 7]. This generality is what makes LLMs both powerful and hard to interpret: a single fixed network comes to encode a vast range of linguistic and world knowledge, distributed across its parameters and activations in a form that no direct reading of the weights makes legible.

2.1.2 THE TRANSFORMER

Modern LLMs are almost universally built on the *transformer* architecture [44]. It embeds the input tokens into a sequence of vectors, one per position, and refines them through a stack of identical *blocks* before reading the final vectors out as next-token distributions. Each block is made of two sublayers with complementary roles: an *attention* sublayer, which exchanges information between positions, and a multi-layer perceptron (MLP) sublayer, which transforms each position on its own. Figure 2.1 shows this organisation.

ATTENTION *Attention* is the mechanism by which positions exchange information. At each position, an *attention head* forms a query and compares it against a key emitted by itself and every position before it; the better a position matches, the more it contributes to a weighted average of the corresponding values, which becomes the head's output for the current position [44]. Concretely, the head projects each input to a query, key, and value, $\boldsymbol{q}_t = Q\boldsymbol{x}_t$, $\boldsymbol{k}_s = K\boldsymbol{x}_s$, and $\boldsymbol{v}_s = V\boldsymbol{x}_s$, and returns

$$\text{head}(\boldsymbol{x}_{1:t}) = \sum_{s=1}^{t} a_{ts}\boldsymbol{v}_s, \qquad a_{ts} = \text{softmax}_s\left(\frac{\boldsymbol{q}_t^\top \boldsymbol{k}_s}{\sqrt{d_k}}\right), \tag{2.1}$$

where $\boldsymbol{x}_{1:t} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_t)$ is the prefix of input states up to position t, d_k is the dimension of the queries and keys, and the softmax runs over $s = 1, \ldots, t$, so that a position attends only to itself and those before it. A sublayer runs several such heads in parallel — each free to specialise, one linking a pronoun to its referent, another a verb to its subject — and combines their outputs by concatenation and a learned output projection. Because attention is the only component that moves information between positions, it is where the model assembles context: it turns a position's representation from a fact about a single token into one that reflects the surrounding text.

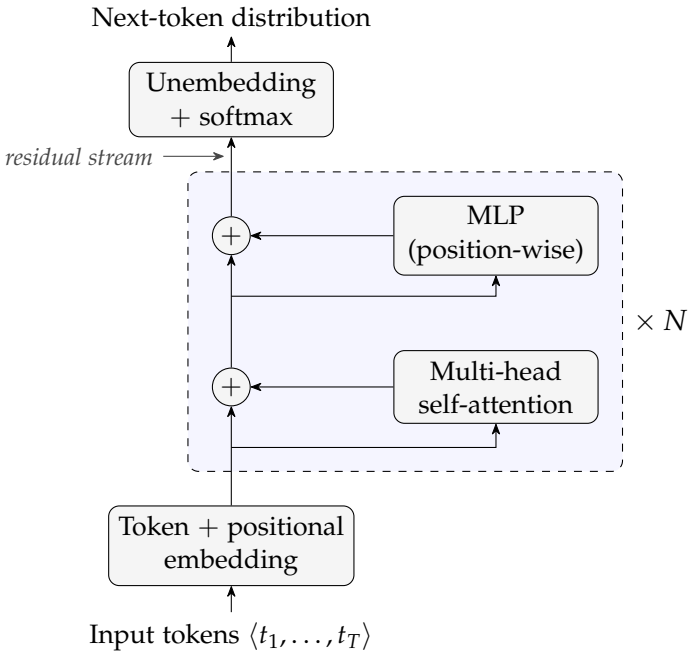

Figure 2.1: A decoder-only transformer, drawn around the *residual stream*. Each token is embedded into a vector; the resulting per-position state flows up the central stream, and every block reads from the stream and adds its result back through a residual connection — first a multi-head self-attention sublayer, which mixes information across positions, then a position-wise MLP sublayer. After N such blocks the final stream state is unembedded into a next-token distribution. The additive structure of eq. (2.3) is what makes the residual stream a single, well-defined activation space to decompose; the view follows Elhage et al. [9].

MLP SUBLAYERS Where attention mixes information across positions, an MLP sublayer processes each position on its own, applying the same feed-forward transformation to the vector at every position independently,

$$\mathrm{MLP}(\boldsymbol{x}_t) = W_2\,\varphi\,(W_1\boldsymbol{x}_t + \boldsymbol{b}_1) + \boldsymbol{b}_2, \tag{2.2}$$

with a pointwise nonlinearity φ, an up-projection W_1 to a wider hidden layer and a down-projection W_2 back. Much of the model's capacity sits in these sublayers, which are widely thought to store and retrieve its learned associations and are a primary site at which interpretable features appear [4].

THE RESIDUAL STREAM A single detail ties these sublayers together. Neither sublayer overwrites the vector it is given; each computes an update and *adds* it back, through a *residual connection*. Every position therefore carries one running vector — the *residual stream* — that every sublayer of every block reads from and writes to. Let $\boldsymbol{x}_t^{(\ell)}$ denote the

residual stream at position t as it enters block ℓ; the block updates it as

$$\boldsymbol{x}_t^{(\ell+1)} = \boldsymbol{x}_t^{(\ell)} + \mathrm{Attn}^{(\ell)}\left(\boldsymbol{x}_{1:t}^{(\ell)}\right) + \mathrm{MLP}^{(\ell)}\left(\boldsymbol{x}_t^{(\ell)}\right). \tag{2.3}$$

The stream starts as the token's embedding and accumulates the output of every sublayer in turn; because attention writes into it, a position's stream is never a fact about that token alone but a running summary of the context around it. This additive structure is what makes the residual stream a single, well-defined space in which to study the model's computation [9]. It is also the object the later chapters take apart: the activation vector $\boldsymbol{x} \in \mathbb{R}^n$ that the autoencoders of section 2.3 receive as input is exactly such a residual stream state $\boldsymbol{x}^{(\ell)}$, read off at a chosen layer ℓ.

2.2 EXPLAINABLE AI

The incredible performance of modern machine-learning systems comes at the cost of transparency: a deep network's output is the product of millions of interacting parameters, with no accompanying account of why it was produced. EXplainable Artificial Intelligence (XAI) is the field that sets out to supply such accounts, making a model's behaviour intelligible enough to be trusted, debugged, or learned from. The methods relevant to this thesis are most naturally organised around a shift in what counts as an explanation: from *attribution*, which asks *where* ("which input features drove a prediction?"), to *concepts*, the human-meaningful units in terms of which a model can be said to reason. It is this latter, concept-based view on which this work builds.

2.2.1 TRADITIONAL XAI

2.2.1.1 *Intrinsically Interpretable Models*

The most direct route to an interpretable model is to choose one whose decision process is transparent by construction. Linear and logistic regression, decision trees, naive Bayes classifiers, and k-nearest neighbours are the canonical examples: their parameters or structure can be inspected directly, so that the explanation *is* the model rather than a separate artefact derived from it [26]. Rudin [35] argues that such models should be preferred wherever they are competitive, since post-hoc explanations of an opaque model are approximations that need not be faithful to its actual computation. This route is, however, closed for the systems studied in this thesis: LLMs comprise billions of nonlinearly interacting parameters, and no direct reading of their weights yields a human-understandable account of their behaviour. Interpretability must therefore be recovered *post hoc*, from a model that is already trained and fixed.

2.2.1.2 *Post-hoc Attribution Methods*

The dominant post-hoc paradigm explains an individual prediction by *attributing* it to the input features, assigning each feature a score that quantifies its contribution to the output. LIME [34] does this by fitting a simple, interpretable surrogate (typically a sparse linear model) to the behaviour of the black box in a local neighbourhood of the instance being explained. SHAP [23] unifies this and several related schemes under the Shapley value from cooperative game theory, yielding feature attributions that are additive and satisfy a set of desirable axioms. For models operating on images, gradient-based saliency maps [38] and their refinements such as Grad-CAM [36] produce a heatmap over the input that highlights the regions most influential to the prediction. What these methods share is also their central limitation: they indicate *where* in the input the model is sensitive, but not *what* the model has internally come to represent. An attribution map can mark a pixel or a token as important without revealing the human-meaningful concept it participates in: a gap that motivates the concept-based methods of section 2.2.2.

COUNTERFACTUAL EXPLANATIONS A counterfactual explanation answers a contrastive question: what is the smallest change to the input that would have produced a different, desired prediction [45]? Rather than scoring the features of the present input, it points to a nearby instance on the other side of the decision boundary: "had this applicant's income been slightly higher, the loan would have been granted". Such explanations are attractive because they are actionable and require no access to the model's internals, only the ability to query it. Generating counterfactuals that are at the same time close to the original input and realistic is itself a modelling problem, one which generative models are naturally suited to solve.

2.2.1.3 *Prototypes*

A complementary, example-based strategy explains a model not through feature scores but through representative instances. A *prototype* is a data point that is typical of a region of the input space, chosen so that a small set of such points summarises the data (or the model's behaviour upon it) in terms a human can grasp by inspection [18]. The defining characteristic, for the purposes of this thesis, is that a prototype in this traditional sense is an *input instance*: an actual or representative example drawn from the data space.

2.2.2 CONCEPT-BASED XAI

Concept-based eXplainable Artificial Intelligence (C-XAI) answers the *what* question that attribution leaves open: instead of pointing to the

inputs responsible for a prediction, it explains the prediction in terms of high-level, human-meaningful *concepts* (“striped”, “has wheels”, “expresses uncertainty”) of the kind a person would use to describe the same decision. Methods in this family differ mainly in whether the concepts are built into the model from the outset or recovered from a model that was trained without them.

2.2.2.1 *Explainable-by-design Models*

One way to guarantee that concepts genuinely take part in a model’s decision is to make them an explicit, intermediate stage of the computation, so that the prediction is forced to pass through a concept representation before reaching the output.

CONCEPT BOTTLENECK MODELS A proposal to insert such a stage comes from Concept Bottleneck Models (CBMs) [22]: the network first maps the input to a set of human-specified concepts and then predicts the label from those concepts alone using a linear, intrinsically interpretable layer. Because the output depends on the input only through the concept layer, the model’s reasoning can be read off the activated concepts, and a user can intervene on them, correcting a mistaken concept and observing the effect on the prediction. The downside is supervision: training a CBM requires data annotated by human experts with concept labels, which is costly and ties the model to a fixed, predefined vocabulary.

CONCEPT EMBEDDING MODELS A known weakness of CBMs is the accuracy–interpretability trade-off: when the predefined concepts do not carry all the information the task requires, forcing the prediction through a narrow concept bottleneck degrades performance. Concept Embedding Models (CEMs) [11] loosen the bottleneck by representing each concept not as a single scalar but as a pair of high-dimensional embeddings, one for its active and one for its inactive state. This recovers accuracy comparable to an unconstrained black box while retaining concept-level interpretability and the possibility of intervention. Like CBMs, these models require a concept-annotated dataset.

LABEL-FREE CBMS Addressing annotation cost directly is the objective of Label-free Concept Bottleneck Models (LF-CBMs) [27]. Rather than relying on a hand-labelled concept set, they extract candidate concepts using an LLM and align the network’s activations with them through a vision–language model such as CLIP, converting a standard backbone into a CBM without any concept supervision. This makes the approach scalable to settings where dense concept annotation would be impractical.

CONCEPT BOTTLENECK LLMS The concept-bottleneck idea has recently been carried over to language. Concept Bottleneck Large Language Models (CB-LLMs) [40] insert a concept bottleneck layer into an LLM, routing text classification — and, in a generative variant, generation itself — through a set of interpretable concepts before the output. As in an LF-CBM, a hand-annotated concept set is not needed: the concepts are generated by a prompted LLM and each training text is scored against them automatically, so the bottleneck is built without concept supervision. The explicit concept layer also makes the model *steerable*, since intervening on a concept predictably shifts its behaviour. Like the other methods in this group, CB-LLMs are *explainable by design*: interpretability is secured by training the model with the concept bottleneck built in.

Even so, that interpretability is only partial. In the generative variant the next-token prediction is shaped not by the concept bottleneck alone but by it together with a parallel layer of unsupervised, uninterpretable neurons added to preserve generation quality. When the dataset labels are reused as concepts the bottleneck can be tiny — 4 neurons for the 4 AG News classes — yet in the authors' implementation it runs alongside 768 unsupervised ones,[1] so only 4 of the 772 dimensions feeding each token prediction carry interpretable concepts. Those interpretable dimensions remain intervenable nonetheless: because they feed the prediction directly, clamping or amplifying one steers the model's behaviour predictably — the steerability the bottleneck is built for — even if they carry only a fraction of the signal.

2.2.2.2 *Post-hoc Methods*

The alternative to *explainable-by-design* models is to leave the trained model untouched and probe it for the concepts it has already come to represent.

TESTING WITH CONCEPT ACTIVATION VECTORS T-CAV [19] represents a concept as a direction in a layer's activation space: given example inputs exhibiting the concept and a set of random ones, it fits a linear classifier to separate them and takes the normal to the decision boundary — the concept activation vector — as the concept's direction. The model's sensitivity to the concept is then quantified by the directional derivative of the output along this vector, yielding a global statement of how much a given concept influences a class prediction. The concept must still be supplied in advance, as a set of examples.

1 The width of the unsupervised layer is fixed at 768 in the authors' released code (https://github.com/Trustworthy-ML-Lab/CB-LLMs); the paper itself leaves it unspecified.

SAES AND MECHANISTIC INTERPRETABILITY A final, unsupervised strand removes even that requirement. Rather than testing for concepts named ahead of time, sparse autoencoders (SAEs) learn a dictionary of concept-like features directly from a model's internal activations [15], and have become a central tool of Mechanistic Interpretability (MechInt) — the effort to reverse-engineer the computations of neural networks, and of large language models in particular. This unsupervised discovery of interpretable features is treated in full in section 2.3.2.

2.3 AUTOENCODERS

An autoencoder is a neural network trained to reconstruct its input through an intermediate latent representation [13, Ch 14]. It consists of two components: an **encoder** $f_\phi : \mathbb{R}^n \to \mathbb{R}^m$ that maps an input $\boldsymbol{x}$ to a latent code $\boldsymbol{z} = f_\phi(\boldsymbol{x})$, and a **decoder** $g_\theta : \mathbb{R}^m \to \mathbb{R}^n$ that reconstructs the input as $\hat{\boldsymbol{x}} = g_\theta(\boldsymbol{z})$. Training minimises a reconstruction loss, typically the mean squared error (MSE),

$$\mathcal{L}_{\text{recon}} = \text{MSE}(\boldsymbol{x}, \hat{\boldsymbol{x}}) = \frac{1}{n}\|\boldsymbol{x} - \hat{\boldsymbol{x}}\|_2^2. \tag{2.4}$$

When $m < n$, the bottleneck forces the model to learn a compressed representation. When $m > n$, the network is overcomplete and additional constraints are required to prevent the trivial solution of learning the identity. In either case, the structure of the latent space is shaped by the choice of architecture and regularisation.

2.3.1 VARIATIONAL AUTOENCODERS

Variational autoencoders (VAEs) [21, 33] reinterpret the autoencoder as a probabilistic generative model. Rather than mapping an input to a single latent point, the encoder maps it to a *distribution* over latent codes, and the decoder is treated as a generative model that defines a distribution over inputs given a latent code. This probabilistic framing turns the autoencoder into a model from which new data can be sampled, and forces its latent space to have a smooth, continuous structure.

This smoothness is the property that sets a VAE apart from a standard autoencoder. Trained for reconstruction alone, an autoencoder is free to scatter its encodings into well-separated clusters with large gaps between them; those gaps are regions that decode to unrealistic outputs, so the space cannot be sampled or interpolated reliably. The VAE objective instead pushes encodings to fill the latent space more uniformly and to vary smoothly, so that nearby codes decode to similar inputs and paths between codes stay on the data manifold. This

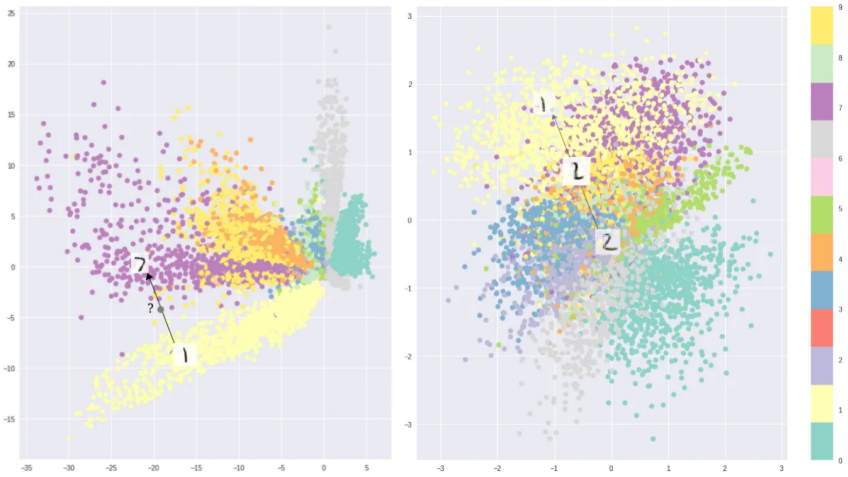

Figure 2.2: Two-dimensional latent spaces learned on MNIST, coloured by digit class, reproduced from Shafkat [37]. Optimising for reconstruction alone (left) lets a standard autoencoder separate the classes into disconnected clusters with gaps that decode to unrealistic samples, whereas the variational autoencoder objective (right) yields a smoother, more continuous arrangement, concentrated around the origin by the prior, that can be sampled and interpolated.

allows the VAE to be used as a generative model. Figure 2.2 illustrates the difference on a two-dimensional latent space trained on MNIST.

2.3.1.1 *Variational Inference*

Formally, a VAE models the data through a generative process in which a latent code is first drawn from a prior $z \sim p(z)$ — typically a standard Gaussian $p(z) = \mathcal{N}(\mathbf{0}, \mathbb{I})$ — and an observation is then drawn from a decoder distribution $p_\theta(\boldsymbol{x} \mid z)$ parameterised by a neural network. Training would ideally maximise the marginal likelihood

$$p_\theta(\boldsymbol{x}) = \int p_\theta(\boldsymbol{x} \mid z)\, p(z)\, \mathrm{d}z\,,$$

but this integral, and the corresponding posterior $p_\theta(z \mid \boldsymbol{x}) = p_\theta(\boldsymbol{x} \mid z) p_\theta(z) / p_\theta(\boldsymbol{x})$, are intractable for any non-trivial decoder.

The key idea of Kingma and Welling [21] is to sidestep this intractability with *amortised variational inference*: an encoder network $q_\phi(z \mid \boldsymbol{x})$ is trained to approximate the true posterior, with all data points sharing the same inference parameters ϕ. The encoder outputs the parameters of a diagonal Gaussian,

$$q_\phi(z \mid \boldsymbol{x}) = \mathcal{N}\Big(z;\, \boldsymbol{\mu}_\phi(\boldsymbol{x}),\, \mathrm{diag}(\boldsymbol{\sigma}^2_\phi(\boldsymbol{x}))\Big)\,, \tag{2.5}$$

so that each input is mapped to a mean and a (diagonal) covariance rather than to a single point.

Instead of the intractable likelihood, the VAE maximises a tractable lower bound on it, the evidence lower bound (ELBO),

$$\log p_\theta(\boldsymbol{x}) \geq \underbrace{\mathbb{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})}\left[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})\right]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\right)}_{\text{regularisation}} \equiv \text{ELBO}. \quad (2.6)$$

The bound follows from the identity

$$\log p_\theta(\boldsymbol{x}) = \text{ELBO} + D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p_\theta(\boldsymbol{z} \mid \boldsymbol{x})\right),$$

together with the non-negativity of the Kullback–Leibler (KL) divergence; it is tight precisely when the approximate posterior matches the true posterior $p_\theta(\boldsymbol{z} \mid \boldsymbol{x})$. Maximising the ELBO simultaneously pushes the decoder to assign high likelihood to the data (the reconstruction term) and keeps the approximate posterior close to the prior (the KL divergence term). Negating it yields the training loss

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\right). \quad (2.7)$$

When the decoder is Gaussian with fixed isotropic variance, the first term reduces, up to an additive constant, to the MSE reconstruction loss of eq. (2.4). divergence between two Gaussians, has a closed form. For the standard-normal prior it becomes

$$D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\right) = \frac{1}{2}\sum_{j=1}^{m}\left(\mu_j^2 + \sigma_j^2 - \log\sigma_j^2 - 1\right),$$

where μ_j and σ_j are the components of the encoder's predicted mean and standard deviation.

Evaluating the reconstruction term requires sampling $\boldsymbol{z}$ from $q_\phi(\boldsymbol{z} \mid \boldsymbol{x})$, an operation that is not differentiable with respect to the encoder parameters and would therefore block gradient-based training. The VAE resolves this with the *reparameterisation trick*: the stochasticity is moved to an auxiliary noise variable, and the sample is expressed as a deterministic, differentiable function of the encoder outputs,

$$\boldsymbol{z} = \boldsymbol{\mu}_\phi(\boldsymbol{x}) + \boldsymbol{\sigma}_\phi(\boldsymbol{x}) \odot \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbb{I}),$$

where $\odot$ denotes the element-wise product. Gradients can now flow through $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}_\phi$ while the randomness is confined to $\boldsymbol{\epsilon}$, making the whole objective trainable end-to-end by stochastic gradient descent.

2.3.1.2 *Architectures*

There exist several variants that modify the base VAE to shape the latent space or the form of the latent code; two are particularly relevant here.

β-VAE Higgins et al. [14] introduced a single modification: a scalar weight β on the KL term of the objective,

$$\mathcal{L}_{\beta\text{-VAE}} = \mathcal{L}_{\text{recon}} + \beta\, D_{\text{KL}}\big(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\big\|\, p(\boldsymbol{z})\big)\,. \tag{2.8}$$

Setting $\beta > 1$ places additional pressure on the approximate posterior to match the factorised prior, which encourages individual latent dimensions to capture statistically independent factors of variation — a property known as *disentanglement*. The standard VAE is recovered at $\beta = 1$. Changing β offers a trade-off: a larger value improves the structure and interpretability of the latent space at the cost of reconstruction fidelity.

VQ-VAE van den Oord, Vinyals and Kavukcuoglu [43] replaced the continuous Gaussian latent with a *discrete* one. The encoder output is quantised to the nearest entry of a learned codebook $\{\boldsymbol{e}_1, \ldots, \boldsymbol{e}_K\}$, so that each latent is represented by a discrete index rather than a continuous vector. This sidesteps the *posterior collapse* that can affect continuous VAEs paired with expressive autoregressive decoders, in which the decoder learns to ignore the latent entirely. Because the nearest-neighbour lookup is non-differentiable, gradients are passed to the encoder with a straight-through estimator and the codebook is learned through dedicated loss terms. The vector-quantised variational autoencoder (VQ-VAE) underpins many modern generative systems, providing the discrete latent vocabulary over which an autoregressive prior is subsequently trained.

2.3.2 SPARSE AUTOENCODERS

Sparse autoencoders (SAEs) are one of the main variants of autoencoders. Just like standard autoencoders, they learn a dictionary of feature directions from the input data, but they typically are overcomplete ($m \gg n$) and their latent representation is forced to be sparse using some kind of sparsity mechanism. Given input $\boldsymbol{x} \in \mathbb{R}^n$, the SAE encodes it as

$$\boldsymbol{z} = \text{ReLU}(W_e\left(\boldsymbol{x} - \boldsymbol{b}_{\text{pre}}\right) + \boldsymbol{b}_e), \tag{2.9}$$

where $W_e \in \mathbb{R}^{m \times n}$ is the encoder weight matrix, $\boldsymbol{b}_e \in \mathbb{R}^m$ is the encoder bias, and $\boldsymbol{b}_{\text{pre}} \in \mathbb{R}^n$ is a pre-encoder bias subtracted from the input before encoding. This learned offset allows the encoder to operate on roughly mean-centred activations, improving training stability [4]. The decoder then reconstructs the input as

$$\hat{\boldsymbol{x}} = W_d \boldsymbol{z} + \boldsymbol{b}_{\text{pre}}, \tag{2.10}$$

where $W_d \in \mathbb{R}^{n \times m}$ is the decoder weight matrix. Each column $\boldsymbol{d}_i$ of W_d is a dictionary atom, representing a candidate feature direction in activation space.

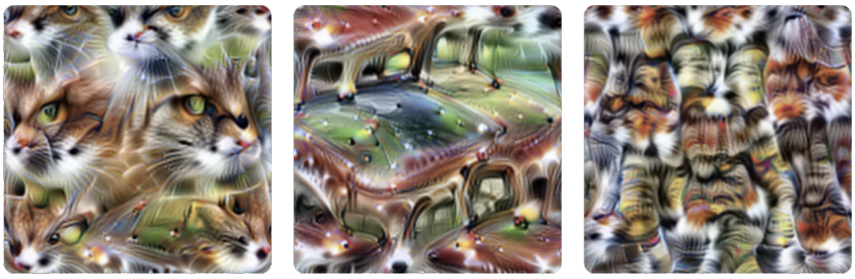

Figure 2.3: Neuron 4e:55, a polysemantic neuron from InceptionV1 which responds to cat faces, fronts of cars, and cat legs. Reproduced from Olah et al. [28].

When used in the context of MechInt, the overcompleteness and the sparsity constraint are motivated by the *superposition hypothesis*.

2.3.2.1 *The Superposition Hypothesis*

A central challenge in MechInt is that individual neurons in neural networks tend to be *polysemantic*: they activate in response to multiple, seemingly unrelated concepts [10, 28]. An example of this can be seen in fig. 2.3. This runs contrary to the intuition that an interpretable model should have neurons with clean, well-defined roles.

Elhage et al. [10] proposed the *superposition hypothesis* to explain this phenomenon. The core idea is that a network may represent more features than it has dimensions by exploiting the fact that natural inputs are sparse: at any given time, only a small fraction of all possible features are relevant. Under this condition, a network can pack many near-orthogonal feature directions into a lower-dimensional space with limited interference; polysemanticity is therefore a rational compression strategy under capacity constraints.

This has a direct implication for interpretability: the natural basis of a model's activations is not the neuron basis, but a larger, overcomplete set of feature directions embedded within it. Recovering this basis requires moving beyond individual neuron analysis, towards methods that can identify sparse, distributed structure. This is precisely the goal of *sparse dictionary learning* [29], a classical technique from computational neuroscience that SAEs adapt to the setting of modern language models.

2.3.2.2 *Architectures*

There are many ways to enforce sparsity, each with its pros and cons. Here is a selection of methods that range from the initial SAE idea to more modern architectures that try to improve on it.

VANILLA ℓ_1 This is the original architecture proposed by Huben et al. [15]. It uses an ℓ_1 penalty[2] on the latent code to encourage most of $\boldsymbol{z}$'s entries to be exactly zero, so that any given activation is reconstructed from a small number of active features. The loss used to train this model is

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \lambda \|\boldsymbol{z}\|_1. \tag{2.11}$$

The scalar λ controls the sparsity–reconstruction trade-off and must be tuned carefully: too large, and reconstruction degrades; too small, and the learned features lose monosemanticity. A known side effect of ℓ_1 regularisation is *shrinkage* [42, 46]: beyond driving inactive features to zero, the penalty also biases active feature magnitudes below their reconstruction-optimal values, causing the model to systematically underestimate the true activation of each feature [31, 46]. Constraining the decoder columns to unit ℓ_2 norm after each gradient step partially mitigates this, by preventing the model from reducing the ℓ_1 penalty through the trivial reparameterisation of scaling down encoder outputs and compensating with larger decoder norms [31]. There is, however, a residual gradient-level bias that is structurally entangled with the ℓ_1 penalty and cannot be resolved with this trick.

TOPK SAES Gao et al. [12] proposed replacing the ℓ_1 penalty with a hard sparsity constraint, implemented via a TopK activation function that retains only the k largest pre-activations and zeros the rest. This directly controls the number of active features per forward pass, eliminating the need to tune λ and removing the shrinkage bias entirely. The full training objective is

$$\mathcal{L} = \mathcal{L}_{\text{recon}} + \alpha \mathcal{L}_{\text{aux}}, \tag{2.12}$$

where the auxiliary loss $\mathcal{L}_{\text{aux}}$ addresses the problem of dead latents, i. e., features that cease to activate entirely during training. Latents are flagged as dead if they have not activated for a predetermined number of tokens. Given the reconstruction residual $\boldsymbol{e} = \boldsymbol{x} - \hat{\boldsymbol{x}}$, the top-$k_{\text{aux}}$ dead latents are selected and used to reconstruct $\boldsymbol{e}$; the resulting reconstruction error is $\mathcal{L}_{\text{aux}}$. This gives dead latents a gradient signal proportional to how much of the residual they could explain, encouraging them to activate rather than remain permanently dormant. Gao et al. [12] demonstrate that TopK SAEs achieve better reconstruction at equivalent sparsity levels compared to ℓ_1 baselines, making them a strong and widely used alternative.

GATED SAES Rajamanoharan et al. [31] introduced an architecture that separates the decision of whether a feature is active from the decision of how strongly it activates. A dedicated gating pathway

2 The ℓ_p norm is defined as $\ell_p(\boldsymbol{x}) = \|\boldsymbol{x}\|_p = \left(\sum_i |x_i|^p\right)^{1/p}$, $p \geq 1$. Throughout, $\|\cdot\| = \|\cdot\|_2$.

produces a binary mask via a Heaviside step function (approximated during training with a straight-through estimator) which is applied element-wise to the encoder output. This architectural separation allows the model to learn feature presence and magnitude more independently, addressing shrinkage from a structural rather than a loss-based perspective.

JUMPRELU SAES Rajamanoharan et al. [32] proposed replacing the standard ReLU with a JumpReLU activation, which is zero below a learned per-feature threshold θ_i and equal to the pre-activation above it,

$$\text{JumpReLU}_{\theta_i}(z_i) = z_i \cdot \mathbb{1}[z_i > \theta_i]. \tag{2.13}$$

The key advantage of this parameterisation is that it provides a dedicated, per-feature learnable threshold, which allows the ℓ_0 norm[3] of the feature activations — $\sum_i H(\pi_i - \theta_i)$, where π_i are the encoder pre-activations and $H(x)$ is the Heaviside step function — to be trained directly via a straight-through estimator applied to the threshold parameter θ_i. Because each threshold can be positioned near the actual boundary between active and inactive pre-activations for its feature, the gradient signal is stable and controllable, making ℓ_0 training practical in a way that would not be possible with a fixed threshold. Training with ℓ_0 rather than ℓ_1 eliminates shrinkage entirely, since ℓ_0 penalises only the number of active features and not their intensity. JumpReLU SAEs achieve comparable performance to TopK SAEs.

2.3.2.3 *Applications to MechInt*

SAEs find their primary application in interpreting the internal activations of neural networks, making MechInt the focus of this section. SAEs can be trained on activations from any layer or component of a transformer (section 2.1.2); common targets include the residual stream at various depths, MLP layer outputs, and attention head outputs [4, 15]. The resulting dictionary then supports the applications described below.

FEATURE DISCOVERY The primary use of SAEs is *feature discovery*: training an SAE on a model's internal activations and examining the resulting dictionary atoms to identify human-interpretable concepts. The underlying hypothesis is that each atom, active for only a semantically coherent set of inputs, corresponds to a monosemantic feature of the model's computation — a clean unit of meaning that the model had obscured in the neuron basis. In practice these features often correspond to recognisable concepts: Bricken et al. [4] trained an SAE on the MLP of a one-layer transformer and found features

3 The ℓ_0 norm is a pseudo-norm defined as the number of non-zero vector components: $\ell_0(\boldsymbol{x}) = \sum_i \mathbb{1}[x_i \neq 0]$.

responsive to specific tokens, syntactic roles, and semantic categories, while at production scale Templeton et al. [41] report features for famous people, countries, emotions, or even biases (like racism, sexism, hatred) and sarcastic praise. These results provide strong empirical support for the superposition hypothesis and suggest that SAEs can serve as a practical lens through which to study what information LLMs represent.

STEERING The features recovered by an SAE are not merely descriptive: because each corresponds to a direction in activation space, its activation can be clamped to an arbitrary value during a forward pass to causally *steer* the model's behaviour. Templeton et al. [41] demonstrated this by amplifying a single feature, causing the model to compulsively reference the Golden Gate Bridge regardless of the input. Such interventions show that the discovered features are causally implicated in the model's computation rather than merely correlated with its inputs, and open a path to controllable generation and targeted safety interventions.

CIRCUIT ANALYSIS Beyond cataloguing individual features, SAEs can serve as the building blocks for reconstructing the *circuits* a model uses to perform a computation, expressing the interactions between features across layers as an interpretable causal graph [25]. More recent work favours *transcoders* — a variant trained to approximate the input–output behaviour of an MLP sublayer with a sparse hidden representation, rather than to reconstruct a single activation — which should disentangle the model's computation more cleanly and yield sparser, more faithful circuits [1, 8].

2.3.2.4 *Evaluation*

Assessing the quality of learned features is non-trivial, as interpretability is inherently subjective. Several proxy metrics are used in practice. *Reconstruction quality* is judged by its effect on the model rather than by the raw reconstruction error: the SAE's output is substituted for the original activations, the model is run forward, and the resulting increase in its next-token CE loss is measured [12]. A faithful SAE leaves this loss almost unchanged, indicating that it has retained the information the model actually uses. *Sparsity* is quantified via the average ℓ_0 norm of the latent codes, i. e., the mean number of active features per token. *Automated interpretability* pipelines use a second language model to generate and score natural-language descriptions of individual features [2], providing a scalable if imperfect proxy for human judgement.

CONCLUSIONS

This chapter has assembled the three strands the thesis builds on: the transformer and its residual stream (section 2.1), whose dense, polysemantic activations are the object to be decomposed; the shift in explainability from attribution to concepts (section 2.2), and the concept-based methods that describe a model's computation in human-meaningful terms; and the autoencoder family (section 2.3), from the variational and discrete latents of VAEs and VQ-VAEs to the overcomplete, sparse dictionaries of SAEs.

A single limitation runs through these concept-based methods. Whether a concept is a unit in the bottleneck of a CBM or an atom in the dictionary of an SAE, it is encoded as a single scalar and can vary only in magnitude. This is what forces the sparsity–reconstruction trade-off at the heart of the SAE literature — a faithful reconstruction needs many active concepts, an interpretable decomposition only a few — and the architectures of section 2.3.2.2 all work within it. CEMs are the exception already met: by giving each concept a vector embedding rather than a scalar, they recover the capacity a scalar bottleneck gives up. Section 3.3 carries that idea into the unsupervised setting of activation decomposition — combining the vector-valued concepts of CEMs with the variational inference and discrete latents developed in section 2.3 — and derives the resulting model from first principles.

3

METHODS

Chapter 2 developed the autoencoder family up to the SAEs, whose scalar concept code trades reconstruction against sparsity. This chapter introduces the thesis's response to that trade-off — the Variational Auto Embedding Encoder (VAEE), an unsupervised model that decomposes an activation into a sparse set of *vector*-valued concepts — and derives it from first principles as a probabilistic generative model.

Section 3.1 states the aim in full: why a scalar concept code is what forces the trade-off, and where the VAEE sits among the C-XAI literature it draws on. Section 3.2 fixes the notation. Section 3.3 then develops the model itself — its probabilistic formulation, from the generative model through the inference procedure and the prototype gate; the training objective, derived as a variational lower bound and turned into a practical loss; and the concept interventions that the explicit mask makes available.

3.1 AIM OF THE WORK

SAEs have become the standard tool for decomposing the activations of an LLM into a dictionary of concept-like features (section 2.3.2). Their reach is bounded, though, by a structural trade-off between sparsity and reconstruction [12]: because each concept enters the reconstruction through a single scalar coefficient, reconstructing an activation faithfully tends to require many concepts active at once, whereas the interpretability that motivates the decomposition rests on keeping only a few active per token [4, 15]. Pushed to a useful sparsity, the reconstruction degrades far enough that the reconstructed activation leaves the data manifold. This is more than an aesthetic flaw: any downstream computation is then performed on an out-of-distribution signal even before any intervention, and when a feature is amplified or ablated the resulting change in behaviour conflates the intended edit with this uncontrolled reconstruction error, so the very analyses the decomposition is built for become less reliable than they appear.

The limitation lies in the representation rather than the training: a scalar can record only how strongly a concept is present. This thesis asks whether giving each concept a *vector* embedding relaxes the trade-off. The idea is familiar from CEMs (section 2.2.2), which replace the scalar concept bottleneck of a CBM with per-concept embeddings and so recover the accuracy a narrow bottleneck gives up [11]. CEMs,

however, are supervised, trained against concept labels. The aim here is to carry the concepts-as-vectors idea into the *unsupervised* setting of activation decomposition, and to do so from first principles, defining a probabilistic generative model whose inference procedure and training objective follow from the model itself rather than being assembled by hand. The starting point is the Learnable Concept-based Model (LCBM) of De Santis et al. [6], an unsupervised concept-based image classifier whose encoder produces a vector embedding per concept, gates each concept by comparing each embedding to a learnable prototype, and reconstructs the input and simultaneously classifies it from the embeddings of the active concepts. LCBM supplied the idea rather than the architecture: this thesis keeps its core structure — concepts as embeddings gated against prototypes and decoded by reconstruction — but rebuilds it from the ground up as a fully variational generative model, giving the concept embeddings a probabilistic latent that LCBM leaves deterministic and dropping the supervised classification task, so that the model reconstructs activations from no labels at all.

The VAEE sits at the intersection of three of the families surveyed in chapter 2. From CEMs it takes the representation of a concept as a high-dimensional embedding rather than a scalar; from VAEs it takes a probabilistic latent and amortised variational inference (section 2.3.1); and from VQ-VAEs it takes the idea of a discrete latent over a learned dictionary of vectors (section 2.3.1.2). The two differ in what is made discrete: a VQ-VAE quantises each input to a single codebook entry, so its latent is a categorical index and the vector passed to the decoder is one of finitely many. The VAEE instead keeps its embeddings continuous, its only discrete latent being the binary mask $\boldsymbol{c}$, which decides *which* prototype-anchored concepts are active rather than what their embeddings are. It is in this sense a probabilistic, multilabel counterpart to vector quantisation: independent Bernoulli gates over continuous embeddings in place of a single hard codebook lookup, keeping the expressivity of a continuous latent while holding the active set sparse.

3.2 NOTATION

The VAEE operates on a single input vector $\boldsymbol{x} \in \mathbb{R}^n$ and decomposes it into a dictionary of K concepts. Where an SAE represents each concept by a single scalar coefficient (section 2.3.2), the VAEE represents it by a d-dimensional embedding. The symbols used throughout the chapter are:

- $\boldsymbol{x} \in \mathbb{R}^n$: the input vector, to be reconstructed;
- K: the number of concepts in the dictionary;
- d: the dimension of each concept embedding;

- $\alpha_k(\boldsymbol{x}) \in [0,1]$: the probability that concept k is active for input $\boldsymbol{x}$;
- $\boldsymbol{c} = (c_1, \ldots, c_K) \in \{0,1\}^K$: the binary *concept-activation mask*, where $c_k = 1$ marks concept k active for the given input;
- $\boldsymbol{\mu}_k(\boldsymbol{x}) \in \mathbb{R}^d$: the encoder's output for concept k, the mean of the embedding $\boldsymbol{z}_k$;
- $\boldsymbol{z}_k \in \mathbb{R}^d$: the *embedding* of concept k for the given input, collected into $\boldsymbol{z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_K] \in \mathbb{R}^{Kd}$;
- $\boldsymbol{h}_k \in \mathbb{R}^d$: the learned *prototype* of concept k, a global parameter shared across all inputs.

As in section 2.3, f_ϕ denotes the encoder and g_θ the decoder, with parameters ϕ and θ respectively.

The word *prototype* needs a caveat, since section 2.2.1.3 used it in a slightly different sense. There a prototype was an *input instance* — an actual data point, typical of a region of the data, chosen to summarise it by inspection. The VAEE's prototypes are similar in their function, as they act as a centroid of sorts for a group of input data, but they live in latent space instead of living in the input space.

3.3 MODEL DEVELOPMENT

3.3.1 PROBABILISTIC FORMULATION

The VAEE is defined as a probabilistic generative model of an observation $\boldsymbol{x}$, built so that a sparse set of concept embeddings accounts for it.

3.3.1.1 *Generative Model*

The VAEE places a joint distribution over the mask $\boldsymbol{c}$, the embeddings $\boldsymbol{z}$, and the observation $\boldsymbol{x}$ that factorises along the generative flow $\boldsymbol{c} \to \boldsymbol{z} \to \boldsymbol{x}$,

$$p(\boldsymbol{c}, \boldsymbol{z}, \boldsymbol{x}) = p(\boldsymbol{c})\, p(\boldsymbol{z} \mid \boldsymbol{c})\, p_\theta(\boldsymbol{x} \mid \boldsymbol{z}). \tag{3.1}$$

Its three factors are specified in turn. First, a binary mask decides whether the concept is present: each c_k is drawn independently from a Bernoulli with a fixed prior activation rate π, shared across concepts,

$$p(c_k) = \mathrm{Bern}\left(c_k;\, \pi\right). \tag{3.2}$$

The mask then represents a *Gated Gaussian* prior over the embedding $\boldsymbol{z}$: an inactive concept ($c_k = 0$) draws its embedding from a Gaussian at the origin, an active one ($c_k = 1$) from a Gaussian centred on the concept's prototype $\boldsymbol{h}_k$,

$$p\left(\boldsymbol{z}_k \mid c_k\right) = \mathcal{N}\left(\boldsymbol{z}_k;\, c_k \boldsymbol{h}_k,\, \sigma_0^2 \mathbb{I}\right), \tag{3.3}$$

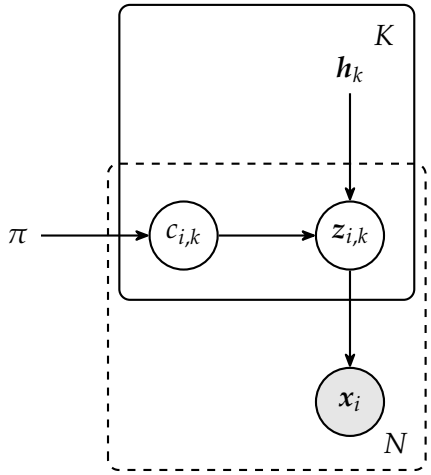

Figure 3.1: The VAEE generative model as a probabilistic graphical model. For each of the K concepts, a binary mask $c_{i,k}$ (drawn from a Bernoulli with rate π) gates a Gaussian embedding $z_{i,k}$ centred either at the origin or at the learned prototype h_k; the gated embeddings generate the observation x_i. Shaded nodes are observed and unshaded are latent, bare symbols are global parameters; the solid plate replicates over concepts and the dashed plate over the N observations.

both with a fixed isotropic variance σ_0^2. The prototype h_k thus has a precise meaning: it is the mean embedding of concept k when the concept is active — its canonical location in embedding space. Finally, the decoder g_θ maps the embeddings to the observation through a Gaussian likelihood centred on its output,

$$p_\theta(x \mid z) = \mathcal{N}(x;\, g_\theta(z),\, \mathbb{I}). \tag{3.4}$$

Figure 3.1 shows this structure as a probabilistic graphical model: the mask fixes where each embedding sits, and the observation follows from the embeddings.

3.3.1.2 *Inference*

Exact inference is intractable: it would require marginalising over the continuous embeddings z and summing over all 2^K possible discrete mask configurations for c,

$$p(x) = \sum_{c} \int p(x, c, z)\, \mathrm{d}z\,.$$

The VAEE instead uses amortised variational inference (section 2.3.1), approximating the true posterior with a *mean-field* family that factorises both over latents and across concepts,

$$q_\phi(c, z \mid x) = \prod_{k=1}^{K} q_\phi(z_k \mid x)\, q_\phi(c_k \mid x), \tag{3.5}$$

with an encoder f_ϕ that predicts the parameters of each factor, shared across all data points. The encoder predicts a mean embedding $\boldsymbol{\mu}_k(\boldsymbol{x})$ for each concept; the embedding posterior is Gaussian about this mean, with the posterior variance frozen to the prior's σ_0^2,

$$q_\phi\left(\boldsymbol{z}_k \mid \boldsymbol{x}\right) = \mathcal{N}\left(\boldsymbol{z}_k;\, \boldsymbol{\mu}_k(\boldsymbol{x}),\, \sigma_0^2 \mathbb{I}\right), \tag{3.6}$$

or, using the reparameterisation trick,

$$\boldsymbol{z}_k = \boldsymbol{\mu}_k(\boldsymbol{x}) + \sigma_0 \boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbb{I}).$$

The activation probability is not predicted by a separate head but *derived* from the predicted embedding location $\boldsymbol{\mu}_k(\boldsymbol{x})$: a concept should be judged active precisely when its predicted embedding lands near its prototype. This is not an arbitrary heuristic but the *optimal* form of the mask factor $q_\phi(c_k \mid \boldsymbol{x})$, dictated by the Gated Gaussian prior of eq. (3.3).

For a mean-field family, the factor that maximises the ELBO while the remaining factors are held fixed has a closed form: its logarithm equals the expectation of the log-joint over those other factors, up to an additive constant [3]. Applied to the mask factor, the expectation runs over every remaining factor, written q_{-k},

$$\begin{aligned}
\log q_\phi^\star(c_k \mid \boldsymbol{x}) &= \mathbb{E}_{q_{-k}}[\log p(\boldsymbol{c}, \boldsymbol{z}, \boldsymbol{x})] + \text{const} \\
&= \mathbb{E}_{q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x})}[\log p(c_k, \boldsymbol{z}_k)] + \text{const} \\
&= \log p(c_k) + \mathbb{E}_{q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x})}[\log p(\boldsymbol{z}_k \mid c_k)] + \text{const}\,.
\end{aligned}$$

Of the joint's factors only those for concept k survive. The other concepts' terms $p(c_j)\, p(\boldsymbol{z}_j \mid c_j)$, $j \neq k$, carry no c_k and pass into the constant; and the likelihood $p_\theta(\boldsymbol{x} \mid \boldsymbol{z})$ of eq. (3.4) conditions on $\boldsymbol{z}$ alone — the generative flow $\boldsymbol{c} \to \boldsymbol{z} \to \boldsymbol{x}$ makes $\boldsymbol{x}$ depend on the mask only through the embeddings — so it too is constant in c_k and absorbed, leaving only the prior $p(c_k)$ and the Gated Gaussian term $p(\boldsymbol{z}_k \mid c_k)$.

Because the mask entry c_k is binary, the factor $q_\phi^\star(c_k \mid \boldsymbol{x})$ is fully characterised by its log-odds: the sigmoid is the inverse of the logit transform,[1] so

$$\begin{aligned}
\alpha_k(\boldsymbol{x}) \equiv q_\phi^\star(c_k = 1 \mid \boldsymbol{x}) &= \text{sigmoid}\left(\log \frac{q_\phi^\star(c_k = 1 \mid \boldsymbol{x})}{1 - q_\phi^\star(c_k = 1 \mid \boldsymbol{x})}\right) \\
&= \text{sigmoid}\left(\log \frac{q_\phi^\star(c_k = 1 \mid \boldsymbol{x})}{q_\phi^\star(c_k = 0 \mid \boldsymbol{x})}\right),
\end{aligned}$$

and the optimality condition above gives those log-odds as a prior term plus an expected likelihood ratio,

$$\log \frac{q_\phi^\star(c_k = 1 \mid \boldsymbol{x})}{q_\phi^\star(c_k = 0 \mid \boldsymbol{x})} = \log \frac{p(c_k = 1)}{p(c_k = 0)} + \mathbb{E}_{q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x})}\left[\log \frac{p(\boldsymbol{z}_k \mid c_k = 1)}{p(\boldsymbol{z}_k \mid c_k = 0)}\right].$$

1 The logit transform is defined as $\text{logit}(s) = \log \frac{s}{1-s}$.

The prior term is the logit of the Bernoulli rate π of eq. (3.2), since $\text{logit}(\pi) = \log \frac{\pi}{1-\pi}$. For the likelihood ratio, substituting the two branches of the Gated Gaussian prior — $\mathcal{N}(\boldsymbol{z}_k;\, \boldsymbol{h}_k,\, \sigma_0^2 \mathbb{I})$ when active and $\mathcal{N}(\boldsymbol{z}_k;\, \boldsymbol{0},\, \sigma_0^2 \mathbb{I})$ when inactive — cancels the shared $(2\pi\sigma_0^2)^{-d/2}$ normalising constants inside the log-ratio, leaving only the exponentials,

$$\begin{aligned} \log \frac{p(\boldsymbol{z}_k \mid c_k = 1)}{p(\boldsymbol{z}_k \mid c_k = 0)} &= \log \frac{\exp\left(-\frac{1}{2\sigma_0^2}\|\boldsymbol{z}_k - \boldsymbol{h}_k\|^2\right)}{\exp\left(-\frac{1}{2\sigma_0^2}\|\boldsymbol{z}_k\|^2\right)} \\ &= \frac{1}{2\sigma_0^2}\left(\|\boldsymbol{z}_k\|^2 - \|\boldsymbol{z}_k - \boldsymbol{h}_k\|^2\right). \end{aligned}$$

Expanding the squared distance $\|\boldsymbol{z}_k - \boldsymbol{h}_k\|^2 = \|\boldsymbol{z}_k\|^2 - 2\,\boldsymbol{z}_k \cdot \boldsymbol{h}_k + \|\boldsymbol{h}_k\|^2$ makes the $\|\boldsymbol{z}_k\|^2$ terms cancel exactly, so the likelihood ratio collapses to an inner product against the prototype that is *linear* in the embedding,

$$\begin{aligned} \mathbb{E}_{q_\phi(\boldsymbol{z}_k|\boldsymbol{x})}\left[\log \frac{p(\boldsymbol{z}_k \mid c_k = 1)}{p(\boldsymbol{z}_k \mid c_k = 0)}\right] &= \mathbb{E}_{q_\phi(\boldsymbol{z}_k|\boldsymbol{x})}\left[\frac{1}{\sigma_0^2}\left(\boldsymbol{z}_k \cdot \boldsymbol{h}_k - \tfrac{1}{2}\|\boldsymbol{h}_k\|^2\right)\right] \\ &= \frac{1}{\sigma_0^2}\left(\boldsymbol{\mu}_k \cdot \boldsymbol{h}_k - \tfrac{1}{2}\|\boldsymbol{h}_k\|^2\right). \end{aligned}$$

Linearity is what makes the expectation tractable, and exactly so: with no term beyond first order in $\boldsymbol{z}_k$, the expectation over $q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x})$ amounts to replacing $\boldsymbol{z}_k$ with its mean $\boldsymbol{\mu}_k(\boldsymbol{x})$. Adding the prior log-odds back and converting through the sigmoid yields the exact activation probability,

$$\alpha_k(\boldsymbol{x}) = \text{sigmoid}\left(\frac{1}{\sigma_0^2}\left(\boldsymbol{\mu}_k(\boldsymbol{x}) \cdot \boldsymbol{h}_k - \tfrac{1}{2}\|\boldsymbol{h}_k\|^2\right) + \text{logit}(\pi)\right). \tag{3.7}$$

In practice the additive constants $-\frac{1}{2\sigma_0^2}\|\boldsymbol{h}_k\|^2 + \text{logit}(\pi)$ are dropped, the scale $1/\sigma_0^2$ folded into a single learnable inverse temperature $1/\tau$, and the inner product generalised to a generic similarity $\text{sim}(\cdot,\cdot)$ (section 3.3.1.3), giving the form used throughout,

$$\alpha_k(\boldsymbol{x}) = \text{sigmoid}\left(\frac{\text{sim}\left(\boldsymbol{\mu}_k(\boldsymbol{x}), \boldsymbol{h}_k\right)}{\tau}\right), \tag{3.8}$$

with $1/\tau$ a single learnable scalar initialised to $\tau = 0.1$. This activation probability parameterises the Bernoulli posterior over the mask,

$$q_\phi\left(c_k \mid \boldsymbol{x}\right) = \text{Bern}\left(c_k;\, \alpha_k(\boldsymbol{x})\right). \tag{3.9}$$

Sampling the mask directly from this Bernoulli is not differentiable with respect to α_k, which would block the gradient from reaching the encoder through the gate. During training the draw is therefore replaced by the reparameterisable *Gumbel-Sigmoid* relaxation, the binary, multilabel analogue of the Gumbel-Softmax (Concrete) distribution

[16, 24]: rather than relaxing a single mutually-exclusive choice over categories, it applies an independent relaxed Bernoulli to each concept, so any subset of concepts may be active at once. A Bernoulli draw can be written as $\mathbb{1}[u < \alpha_k(\boldsymbol{x})] = \mathbb{1}[\operatorname{logit}\alpha_k(\boldsymbol{x}) - g > 0]$ with logistic noise $g = \operatorname{logit}(u)$, $u \sim \text{Uniform}(0,1)$;[2] replacing the indicator with a temperature-controlled sigmoid gives the differentiable surrogate

$$\tilde{c}_k = \text{sigmoid}\left(\frac{\operatorname{logit}\alpha_k(\boldsymbol{x}) - g}{\tau_g}\right). \tag{3.10}$$

The relaxation temperature τ_g sets the sharpness: as $\tau_g \to 0$ the sigmoid tends to the indicator and $\tilde{c}_k$ recovers an exact $\{0,1\}$ Bernoulli sample, while a larger τ_g gives smoother values that keep the gradient well-behaved. At inference the stochastic draw is dropped and both latents are replaced by their posterior means — the embedding $\boldsymbol{z}_k$ by $\boldsymbol{\mu}_k$ and the mask c_k by its expectation $\alpha_k(\boldsymbol{x})$. The gate is then nominally soft, but in practice effectively hard: the binarisation pressure of the training objective (section 3.3.2) drives the learned activations to a strongly bimodal distribution concentrated near 0 and 1 (section 4.5.1.2).

One subtlety remains. The embedding posterior of eq. (3.6) does not depend on the mask, since the encoder produces $\boldsymbol{\mu}_k$ from $\boldsymbol{x}$ alone, so a sampled $\boldsymbol{z}_k$ would pass signal to the decoder regardless of whether the concept is active. The mask $\boldsymbol{c}$ is used to zero each inactive concept before decoding,

$$\hat{\boldsymbol{x}} = g_\theta(\boldsymbol{c} \odot \boldsymbol{z}), \qquad (\boldsymbol{c} \odot \boldsymbol{z})_k = c_k \boldsymbol{z}_k \tag{3.11}$$

This is an architectural choice rather than a term of the variational bound (section 3.3.2), whose likelihood (eq. (3.4)) decodes from the ungated $\boldsymbol{z}$; its primary purpose is to expose the mask $\boldsymbol{c}$ as a clean handle for concept-level interventions (section 3.3.3), where forcing c_k to 0 or 1 switches a concept off or on. Ideally, the gate barely affects an ordinary forward pass: an active concept ($c_k = 1$) passes through untouched, while an inactive one ($c_k = 0$) zeroes an embedding that the conditional-embedding penalty of section 3.3.2 already tries to keep small (section 4.5.1.2). The gate also helps with this: in training, zeroing the inactive slots removes the reconstruction gradient that would otherwise try to inflate their embeddings, leaving the penalty free to hold them near the origin.

2 This logistic noise is the difference of two standard Gumbel variates, which is what ties the relaxation to the Gumbel-Softmax family.

3.3.1.3 *Similarity*

The activation probability of eq. (3.8) is written for a generic similarity $\mathrm{sim}(\cdot,\cdot)$, for which three choices are considered,

$$\mathrm{sim}\left(\boldsymbol{\mu}_k, \boldsymbol{h}_k\right) = \begin{cases} \frac{\boldsymbol{\mu}_k \cdot \boldsymbol{h}_k}{\|\boldsymbol{\mu}_k\| \, \|\boldsymbol{h}_k\|} & \text{(cosine similarity)} \\ \boldsymbol{\mu}_k \cdot \boldsymbol{h}_k & \text{(inner product)} \\ b - \|\boldsymbol{\mu}_k - \boldsymbol{h}_k\| & \text{(negative Euclidean)} \end{cases} . \tag{3.12}$$

Because the embeddings are decoded unnormalised, the choice determines whether the activation probability is coupled to the embedding's magnitude. The exact gate of eq. (3.7) uses the plain inner product, whose decision boundary $\alpha_k = \frac{1}{2}$ sits where $\boldsymbol{\mu}_k \cdot \boldsymbol{h}_k = \frac{1}{2}\|\boldsymbol{h}_k\|^2$; this is well calibrated only as long as $\|\boldsymbol{\mu}_k\| \approx \|\boldsymbol{h}_k\|$, since otherwise the network can inflate $\|\boldsymbol{\mu}_k\|$ to push the inner product up and force a concept active, entangling the discrete gate with a magnitude the decoder also reads. *Cosine similarity* removes this coupling by construction: normalising by $\|\boldsymbol{\mu}_k\| \, \|\boldsymbol{h}_k\|$ leaves the activation probability dependent only on the angle between $\boldsymbol{\mu}_k$ and $\boldsymbol{h}_k$, so the angle controls the gate while the magnitude is left free to carry information to the decoder.[3] The negative Euclidean distance fits the Gaussian prior most directly but requires learning the radius offset b. Cosine similarity is used throughout.

3.3.2 TRAINING OBJECTIVE

The VAEE is trained by maximising a lower bound on the marginal log-likelihood of the data, just as the VAE of section 2.3.1. The marginal likelihood requires marginalising over both latents, an integral over the continuous $\boldsymbol{z}$ and a sum over the 2^K configurations of $\boldsymbol{c}$ that is intractable. Introducing the approximate mean-field posterior of

3 Normalising the similarity does make the temperature essential: a cosine is bounded to $[-1,1]$, so at $1/\tau = 1$ the activation probability could range only over $\mathrm{sigmoid}([-1,1]) \approx [0.27, 0.73]$ and never approach a hard decision. The learnable $1/\tau$ — initialised at 10 (eq. (3.8)) — stretches the bounded cosine onto a range over which the sigmoid can saturate.

eq. (3.5) and applying Jensen's inequality[4] to the concave logarithm gives a lower bound,

$$\begin{aligned}\log p_\theta(\boldsymbol{x}) &= \log \sum_{\boldsymbol{c}} \int p(\boldsymbol{x}, \boldsymbol{c}, \boldsymbol{z})\, \mathrm{d}\boldsymbol{z} \\ &= \log \sum_{\boldsymbol{c}} \int p(\boldsymbol{x}, \boldsymbol{c}, \boldsymbol{z}) \frac{q_\phi(\boldsymbol{c}, \boldsymbol{z} \mid \boldsymbol{x})}{q_\phi(\boldsymbol{c}, \boldsymbol{z} \mid \boldsymbol{x})}\, \mathrm{d}\boldsymbol{z} \\ &= \log \mathbb{E}_{q_\phi(\boldsymbol{c}, \boldsymbol{z} \mid \boldsymbol{x})} \left[\frac{p(\boldsymbol{x}, \boldsymbol{c}, \boldsymbol{z})}{q_\phi(\boldsymbol{c}, \boldsymbol{z} \mid \boldsymbol{x})} \right] \\ &\geq \mathbb{E}_{q_\phi(\boldsymbol{c}, \boldsymbol{z} \mid \boldsymbol{x})} \left[\log \frac{p(\boldsymbol{x}, \boldsymbol{c}, \boldsymbol{z})}{q_\phi(\boldsymbol{c}, \boldsymbol{z} \mid \boldsymbol{x})} \right] \equiv \text{ELBO}.\end{aligned}$$

This mean-field factorisation is the model's central variational approximation: the true posterior couples the mask and its embedding through the generative $\boldsymbol{c} \to \boldsymbol{z}$ edge, so that even given $\boldsymbol{x}$ the two remain dependent. Treating them as conditionally independent given $\boldsymbol{x}$ is what lets all three terms below close in form.

The generative model factorises over concepts as $p(\boldsymbol{x}, \boldsymbol{c}, \boldsymbol{z}) = p_\theta(\boldsymbol{x} \mid \boldsymbol{z}) \prod_k p(c_k)\, p(\boldsymbol{z}_k \mid c_k)$ (eq. (3.1)). Substituting this along with the mean-field posterior and splitting the logarithm of the resulting product gives

$$\text{ELBO} = \mathbb{E}_{q_\phi(\boldsymbol{c}, \boldsymbol{z} \mid \boldsymbol{x})} \left[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z}) + \sum_{k=1}^{K} \log \frac{p(c_k)}{q_\phi(c_k \mid \boldsymbol{x})} + \sum_{k=1}^{K} \log \frac{p(\boldsymbol{z}_k \mid c_k)}{q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x})} \right].$$

Under the mean-field posterior the joint expectation factorises as $\mathbb{E}_{q_\phi(\boldsymbol{c}, \boldsymbol{z} \mid \boldsymbol{x})} = \mathbb{E}_{q_\phi(\boldsymbol{z} \mid \boldsymbol{x})} \mathbb{E}_{q_\phi(\boldsymbol{c} \mid \boldsymbol{x})}$, and the three summands separate. The first carries no mask, so the expectation over $\boldsymbol{c}$ is trivial and it reduces to the reconstruction term

$$\mathbb{E}_{q_\phi(\boldsymbol{z} \mid \boldsymbol{x})} [\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})] .$$

The second carries no embedding, so the expectation over $\boldsymbol{z}$ is trivial and the expectation over $\boldsymbol{c}$ collects into the negative KL between the Bernoulli posterior and prior,

$$\sum_{k=1}^{K} \mathbb{E}_{q_\phi(c_k \mid \boldsymbol{x})} \left[\log \frac{p(c_k)}{q_\phi(c_k \mid \boldsymbol{x})} \right] = - \sum_{k=1}^{K} D_{\mathrm{KL}} \left(q_\phi(c_k \mid \boldsymbol{x}) \,\|\, p(c_k) \right),$$

the mask-prior KL. In the third summand the inner expectation over $\boldsymbol{z}_k$ is itself a KL,

$$\mathbb{E}_{q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x})} \left[\log \frac{p(\boldsymbol{z}_k \mid c_k)}{q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x})} \right] = -D_{\mathrm{KL}} \left(q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_k \mid c_k) \right),$$

4 Given X an integrable real-valued random variable and φ a concave function, $\varphi(\mathbb{E}[X]) \geq \mathbb{E}[\varphi(X)]$.

a function of the still-free c_k, whose outer expectation gives the conditional-embedding KL term

$$-\sum_{k=1}^{K} \mathbb{E}_{q_\phi(c_k|\boldsymbol{x})}\left[D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_k \mid c_k)\right)\right].$$

Collecting the three terms gives the ELBO as the sum of a reconstruction term and the two negative KLs,

$$\begin{aligned}\log p_\theta(\boldsymbol{x}) \geq\ & \underbrace{\mathbb{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})}[\log p_\theta(\boldsymbol{x} \mid \boldsymbol{z})]}_{\text{reconstruction}} \\ & - \sum_{k=1}^{K} \underbrace{D_{\mathrm{KL}}\left(q_\phi(c_k \mid \boldsymbol{x}) \,\|\, p(c_k)\right)}_{\text{mask-prior KL}} \\ & - \sum_{k=1}^{K} \underbrace{\mathbb{E}_{q_\phi(c_k|\boldsymbol{x})}\left[D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_k \mid c_k)\right)\right]}_{\text{conditional-embedding KL}}.\end{aligned} \tag{3.13}$$

The three terms read directly off the generative model. Under the Gaussian decoder of eq. (3.4) the reconstruction term reduces, up to an additive constant, to the MSE of eq. (2.4); the mask-prior KL pulls each concept's activation toward the Bernoulli prior rate π; and the conditional-embedding KL keeps each embedding near its prototype when the concept is active and near the origin when it is not.

The conditional-embedding KL admits a closed form that the practical loss uses directly, derived in appendix A,

$$\begin{aligned}\sum_{k=1}^{K} \mathbb{E}_{q_\phi(c_k|\boldsymbol{x})}\left[D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_k \mid c_k)\right)\right] = \\ = \frac{1}{2\sigma_0^2} \sum_{k=1}^{K} \left[\alpha_k \|\boldsymbol{\mu}_k - \boldsymbol{h}_k\|^2 + (1-\alpha_k)\|\boldsymbol{\mu}_k\|^2\right].\end{aligned} \tag{3.14}$$

3.3.2.1 *From Bound to Objective*

Turning the ELBO into a practical loss takes a few deliberate departures from a strict negated bound, each motivated by interpretability or training stability rather than mathematical necessity.

BATCH-WISE SPARSITY Conventional sparsity penalties — the ℓ_1 and TopK constraints of section 2.3.2 — act along the concept axis, bounding how many concepts a *single sample* may activate. Applied per sample, the mask-prior KL would do the same, pulling every input's $\alpha_k(\boldsymbol{x})$ toward π and so holding every sample to one fixed activation budget. Sparsity is instead constrained along the *batch* axis [6]: rather than how many concepts a sample uses, the penalty limits how often a *given* concept fires across a batch. The per-sample term

is replaced by a single KL between Bernoullis that matches each concept's batch-average activation $\bar{\alpha}_k = \frac{1}{B}\sum_i \alpha_k(\boldsymbol{x}_i)$ to the prior rate,

$$\beta \sum_{k=1}^{K} D_{\mathrm{KL}}(\mathrm{Bern}(\bar{\alpha}_k) \,\|\, \mathrm{Bern}(\pi)) \equiv \beta \sum_{k=1}^{K} D_{\mathrm{KL}}(\bar{\alpha}_k \,\|\, \pi)\,,$$

with B the batch size and β the term's weight; matching $\bar{\alpha}_k$ to π lets concept k fire on about a π fraction of the batch.

A per-sample budget is not as good a target because inputs differ in complexity: some are explained by a few generative features, others by many. After shuffling, a batch mixes both, and it is unlikely that many of its samples all call for the same concept. Constraining each concept's firing rate rather than each sample's count therefore leaves the per-sample count free, letting the model spend many concepts on a complex input and few on a simple one, while still keeping any single concept from firing everywhere or dying out across the dataset without the need of an auxiliary loss to revive dead latents.

ENTROPY BINARISATION Batch-wise sparsity does not by itself force per-sample decisions to be sharp: activations clustered around π would satisfy it while remaining soft. A per-sample binary-entropy penalty is added,

$$\lambda_{\mathrm{ent}}\,\mathcal{H}(\alpha_{i,k}) = -\lambda_{\mathrm{ent}}\left[\alpha_{i,k}\log\alpha_{i,k} + (1-\alpha_{i,k})\log(1-\alpha_{i,k})\right],$$

which is maximal at $\alpha = \frac{1}{2}$ and zero at the binary extremes, so minimising it drives each gate toward $\{0,1\}$ and aligns the soft training-time activation with the binary mask of the generative model.

STOP-GRADIENT ON THE ACTIVATION The conditional-embedding KL couples the activation probability to the embeddings: in closed form (eq. (3.14)) it reads $\alpha_{i,k}\left\|\boldsymbol{\mu}_{i,k} - \boldsymbol{h}_k\right\|^2 + (1-\alpha_{i,k})\left\|\boldsymbol{\mu}_{i,k}\right\|^2$, which the network could shrink by driving $\alpha_{i,k} \to 0$ irrespective of reconstruction. A stop-gradient $\mathrm{sg}(\cdot)$ operator on α inside this term removes that shortcut, so the activation probability is shaped only by the reconstruction and sparsity terms.

SCALE-INVARIANT REDUCTIONS Each term is averaged over its dimensions — the batch B, the concepts K, and the relevant vector dimension (the input n or the embedding d) — so a single set of weights $\gamma, \beta, \lambda_{\mathrm{ent}}$ transfers across architectures of different sizes without rescaling.

3.3.2.2 *Practical Loss*

Collecting these choices and substituting the closed-form conditional-embedding KL of eq. (3.14) gives the objective minimised in training,

$$\mathcal{L} = \frac{1}{Bn}\sum_{i=1}^{B}\|\boldsymbol{x}_i - \hat{\boldsymbol{x}}_i\|^2 + \frac{\beta}{K}\sum_{k=1}^{K} D_{\mathrm{KL}}(\bar{\alpha}_k \,\|\, \pi) + \frac{\lambda_{\mathrm{ent}}}{BK}\sum_{i=1}^{B}\sum_{k=1}^{K}\mathcal{H}(\alpha_{i,k}) + \frac{\gamma}{BKd}\sum_{i=1}^{B}\sum_{k=1}^{K}\left(\mathrm{sg}(\alpha_{i,k})\left\|\boldsymbol{\mu}_{i,k} - \boldsymbol{h}_k\right\|^2 + (1-\mathrm{sg}(\alpha_{i,k}))\left\|\boldsymbol{\mu}_{i,k}\right\|^2\right), \tag{3.15}$$

where $\hat{\boldsymbol{x}}_i = g_\theta(\boldsymbol{c}_i \odot \boldsymbol{z}_i)$ is the gated reconstruction of eq. (3.11) and $\boldsymbol{\mu}_{i,k} = \boldsymbol{\mu}_k(\boldsymbol{x}_i)$ and $\alpha_{i,k} = \alpha_k(\boldsymbol{x}_i)$. The four terms are, in order, the reconstruction error, the batch-wise sparsity KL, the entropy binarisation penalty, and the conditional-embedding KL (active concepts pulled to their prototype, inactive ones to the origin), whose factor $\frac{1}{2\sigma_0^2}$ has been absorbed into the γ hyperparameter.

3.3.3 CONCEPT INTERVENTIONS

Holding the embeddings in gated slots makes the mask $\boldsymbol{c}$ a direct handle on the decomposition, and so a means of intervening on it. Two operations follow immediately. *Removal* sets $c_k \leftarrow 0$, forcing slot k to contribute exactly nothing to the reconstruction regardless of its embedding. *Insertion* sets $c_k \leftarrow 1$, which also requires a value for z_k: the encoder's $\boldsymbol{\mu}_k(\boldsymbol{x})$ is unsuitable, since for a concept judged inactive the conditional-embedding term has pulled it toward the origin rather than the prototype (section 3.3.2). The value consistent with the generative model under $c_k = 1$ is a draw from the active prior $z_k \sim \mathcal{N}(\boldsymbol{h}_k, \sigma_0^2\mathbb{I})$, or, where a reproducible edit is needed, its mean $z_k \leftarrow \boldsymbol{h}_k$.

This is where a sparse, vector-valued code offers an advantage over a scalar dictionary. Editing an SAE reconstruction means adjusting the many features active on a token at once, and moving several scalar coefficients together risks pushing the reconstruction off the data manifold, the failure that motivates the architecture (section 3.1); the VAEE instead edits one prototype-anchored concept at a time, an edit that is both localised and, because it targets a learned mode of the prior, in-distribution by construction. The intervention is approximate in one respect: the remaining slots $j \neq k$ keep their encoded means $\boldsymbol{\mu}_j(\boldsymbol{x})$ rather than being recomputed under the edit, a simplification it shares with concept-bottleneck interventions [22]. The construction supports these operations, but evaluating them — whether a single-concept edit produces a clean, predictable change in the decoded activation — is left to future work (section 5.2).

CONCLUSIONS

This chapter developed the VAEE from the limitation that motivates it. Starting from the observation that an SAE's scalar concept code is what forces the sparsity–reconstruction trade-off (section 3.1), it gave each concept a vector embedding gated against a learned prototype and cast the construction as a probabilistic generative model (section 3.3.1): a binary mask switches each concept on or off, an active concept draws its embedding near the concept's prototype, and a decoder reconstructs the activation from the gated embeddings. Amortised mean-field inference supplies the encoder and the closed-form similarity activation probability, and a variational lower bound on the marginal log-likelihood supplies the training objective (section 3.3.2); a few deliberate departures from that bound — batch-wise sparsity, entropy binarisation, and a stop-gradient on the activation probability — turn it into the practical loss the model minimises. The explicit mask, finally, makes single-concept interventions available as a built-in operation (section 3.3.3).

What the construction does not yet establish is whether it delivers. Relaxing the sparsity–reconstruction trade-off is an empirical claim, and the interventions the mask supports are only as useful as the concepts the model actually learns. Chapter 4 puts both to the test, comparing the VAEE against a parameter-matched SAE on the decomposition of LLM activations.

4

EXPERIMENTS

Chapter 3 derived the VAEE as a probabilistic generative model but left its empirical behaviour open. This chapter puts the architecture's central claim to the test — that representing a concept by a vector rather than a scalar relaxes the sparsity–reconstruction trade-off that bounds SAEs (section 3.1) — on one application of it: the decomposition of LLM activations.

Section 4.1 describes the activation dataset, drawn from the residual stream of a pretrained LLM. Section 4.2 introduces the SAE baseline against which the VAEE is compared at matched parameter count. Section 4.3 sets out the evaluation metrics — a measure of reconstruction fidelity and two of sparsity. Section 4.4 reports the training setup, and section 4.5 presents the quantitative, qualitative, and ablation results.

4.1 EXPERIMENT SETUP

The VAEE operates on a single activation vector $\boldsymbol{x} \in \mathbb{R}^n$ (section 3.2), so evaluating it on a language model requires a corpus of such vectors. These were extracted from the residual stream of `Mistral 7B v0.1` [17], a pretrained decoder-only LLM, run over SetFit's version[1] of the Stanford Sentiment Treebank (SST-2) sentiment classification dataset [39].

SST-2 is a dataset of movie reviews, each with a label (positive/negative) attached. Only the reviews were of interest here, as a corpus of natural language, so the labels were discarded. SST-2 is a small corpus: large enough to exercise reconstruction, but, in hindsight, probably too narrow for a large and varied concept dictionary to emerge, a limitation returned to when the results are interpreted (section 4.5). This problem was exacerbated by the use of SetFit's version of the dataset rather than the original — a choice made to follow the CB-LLM of Sun et al. [40], which uses the same version — since SetFit's training set contains only 6 920 reviews against the original's 67 300. The total number of tokens in each set is shown in table 4.1.

Each review was passed through the model and its residual-stream state was extracted after the final transformer block, just before the classifier head. This yielded the residual-stream representation of every token in the sentence. The resulting activations have dimension $n = 4\,096$, i. e., Mistral's residual-stream dimension.

1 Available for download at https://huggingface.co/datasets/SetFit/sst2.

SET	NO. REVIEWS	NO. TOKENS
Training	6 920	179 146
Validation	872	22 887

Table 4.1: The distribution of reviews and tokens across the training and validation sets of `SetFit/SST-2`.

4.2 BASELINES

The VAEE is compared against a TopK SAE (section 2.3.2), a standard model for MechInt. Its top-k activation fixes the number of active features per sample directly through k, so the reconstruction error can be read against a sparsity level that is set rather than induced by a penalty weight. Sweeping k traces the baseline's sparsity–reconstruction frontier, against which the VAEE's operating point is placed.

The two models are matched on the size of their encoder and decoder rather than on dictionary size, so that any difference reflects how each represents a concept and not how much capacity it is given. Both map the activation through an encoder into a latent code and back through a decoder; that code has m entries for an SAE and $K \cdot d$ for a VAEE — its K concepts of embedding dimension d, written $K \times d$ — and the encoder and decoder weight matrices are sized to match. Matching therefore sets $K \cdot d = m$: the 128×64 VAEE is paired with the $m = 8\,192$ SAE, 64×64 with $m = 4\,096$, and 32×64 with $m = 2\,048$.[2] The SAE's dictionary is correspondingly far larger than the VAEE's concept count — each VAEE concept spends d parameters where an SAE atom spends one.

4.3 EVALUATION METRICS

Two properties are measured: the faithfulness of the model's reconstruction, and the sparsity of its learned concept representation. Reconstruction is reported as a single error metric; sparsity is reported in two ways, because the VAEE and the SAE do not count activity in the same unit.

2 This equalises the encoder and decoder weights, which dominate the parameter count ($\approx 2n \cdot Kd$ for both models). Beyond them the VAEE carries a handful of parameters with no SAE counterpart — its K learnable prototypes, of dimension d, and the gate temperature — so exact matching of the total would pair the three VAEE sizes with SAEs of $m = 8\,194$, $4\,097$, and $2\,049$ rather than the powers of two used here. The surplus is one or two dictionary atoms, under 0.05% of the count, so the totals agree to well within a percent.

4.3.1 RECONSTRUCTION

Reconstruction fidelity is measured by the MSE between an activation and its reconstruction, averaged across the whole dataset, so that lower values mean a more faithful reconstruction. This is the same quantity defined in eq. (2.4). It is the natural choice here: under the Gaussian decoder of eq. (3.4) the VAEE's reconstruction term reduces to exactly this error (section 3.3.2), and it is the standard reconstruction measure for SAEs, so the two models are scored on the objective each is trained to optimise.[3]

Reconstruction fidelity matters because it measures how much of the activation the concept code actually captures: a decomposition that reconstructs its input poorly has discarded part of the signal it was meant to explain, so any concept read off it describes the model only in part. Low reconstruction error is a necessary — if not sufficient — condition for the recovered concepts to be faithful to the representation they decompose.

4.3.2 SPARSITY

A low active-concept count is not the same as a sparse code. The VAEE targets the number of concepts active per sample κ, not the fraction of its dictionary left idle, so at higher κ it may fire a sizable share of its concepts without contradicting its aim. Dictionary-fraction sparsity is the SAE's design principle, not the VAEE's; the quantity that bears on the comparison is the absolute number of active units, reported below in two ways. Both are per-sample counts, and the figures reported are their averages over the N activation vectors in the validation set.

NUMBER OF ACTIVE CONCEPTS κ From an interpretability standpoint, the central sparsity metric is the number of active concepts: this is the number of units a human must understand in order to explain the input instance. Fewer is therefore better, but only up to a point: a code that fires too few concepts cannot describe the input, so the useful range is a small handful of concepts per sample (roughly 2 to 10). For a TopK SAE this corresponds to its k hyperparameter by construction;[4] for a VAEE, $\kappa = \#\{k : \alpha_k(\boldsymbol{x}) \geq 1/2\}$, the count of concepts whose posterior gate exceeds a threshold. Although the inference gate is continuous (section 3.3.1.2), the threshold introduces no real arbitrariness, because the learned activations are strongly bimodal

3 The two reconstruction checks that are standard for SAEs spliced into a language model — the CE loss and perplexity of the patched model — are not reported for this comparison; MSE is the only reconstruction metric on which both models are evaluated here.

4 Technically, TopK selects the k largest activations, but it is possible that after the ReLU only $k' < k$ activations are non-zero, so $\ell_0 = \kappa = k' < k$. In practice, this almost never happens, but the ℓ_0 is recomputed every time for maximum accuracy nonetheless.

METRIC	SAE	VAEE
κ (active concepts)	$\#\{k : z_k(\boldsymbol{x}) > 0\}$	$\#\{k : \alpha_k(\boldsymbol{x}) \geq 1/2\}$
ℓ_0 (active neurons)	κ	$\kappa \cdot d$

Table 4.2: Sparsity metric formulations for SAE and VAEE.

— concentrated near 0 and 1, with little mass in between, as quantified in section 4.5 — so the count is insensitive to the exact cut-off, and $1/2$ was chosen as a sensible threshold for an activation probability.

ℓ_0 COUNT The ℓ_0 pseudo-norm represents the amount of information the decoder has access to in order to reconstruct the input. This amount corresponds to the number of non-zero scalars from which the model can reconstruct. For a TopK SAE, this number is its k hyperparameter by construction;[5] for a VAEE it is the number of active concepts κ multiplied by the embedding dimension of each concept d.

For a fair comparison it is important to consider both metrics, since a VAEE that activates fewer *concepts* than an SAE may still activate more *neurons* per sample. κ is therefore the fair unit for comparing the two models on interpretability, while ℓ_0 is the unit of choice for comparing decoder capacity. Table 4.2 recaps the formulas used to compute κ and ℓ_0 for both models.

4.4 MODEL TRAINING

Unless stated otherwise, the VAEE uses the shallow configuration of section 4.5.3: a single linear-plus-GELU layer for the encoder f_ϕ and a linear decoder g_θ, with cosine similarity for the gate (eq. (3.12)). The embedding posterior standard deviation is fixed at $\sigma_0 = 0.1$ during training; at inference time the sampling is dropped and the embedding $\boldsymbol{z}_k$ is set to its mean value $\boldsymbol{\mu}_k$. The Gumbel-Sigmoid relaxation (eq. (3.10)) uses temperature $\tau_g = 0.5$ during training; at inference the relaxation is dropped and the gate c_k set to its posterior mean α_k.

The VAEE is trained to minimise the objective of eq. (3.15) — reconstruction error, batch-wise sparsity KL, entropy binarisation, and the conditional-embedding KL. The loss weights are set to $\beta = 4$, $\lambda_{\text{ent}} = \gamma = 1$. The Bernoulli prior rate π is the VAEE's sparsity parameter, swept to trace its frontier. It controls each concept's activation rate across a batch, so the product $\tilde{\kappa} = \pi K$ estimates the number of concepts active per sample; π was chosen using this estimate as a target. The final per-sample count κ exceeds the nominal $\tilde{\kappa}$, since reconstruction drives κ towards higher values.

5 See footnote 4.

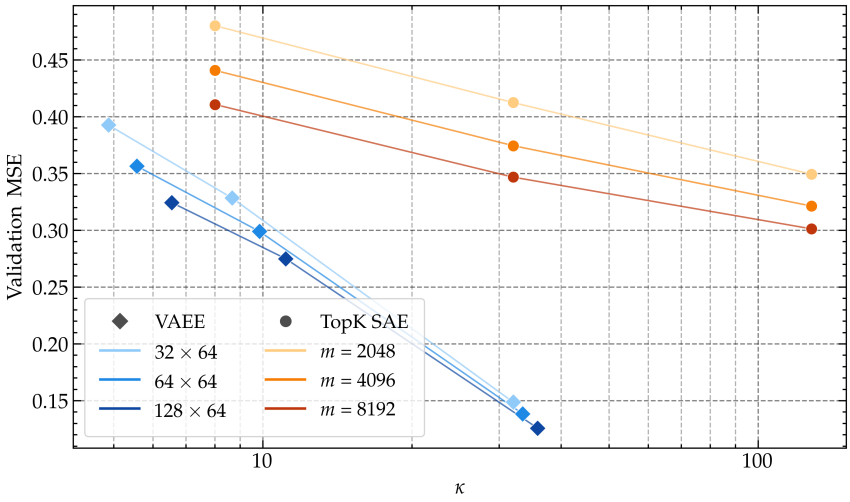

Figure 4.1: Sparsity–reconstruction frontier in active concepts κ, the unit relevant to interpretability; each model sweeps its sparsity parameter at matched parameter count — the SAE over $k \in \{8, 32, 128\}$ and the VAEE over $\tilde{\kappa} = \pi K \in \{4, 8, 32\}$. All VAEE frontiers lie below every SAE frontier — with the VAEE matching the best SAE's reconstruction at far fewer active concepts.

The TopK SAE baseline is trained with its standard objective — reconstruction error under the top-k activation plus auxiliary loss — on the same activations.

Both models were trained using the `Adam` optimiser [20] with a learning rate of 10^{-4} for 3 000 epochs and an early stopping patience of 500 epochs.

4.5 RESULTS

4.5.1 QUANTITATIVE RESULTS

4.5.1.1 *Sparsity–Reconstruction Frontier*

Figure 4.1 plots validation MSE against the number of active concepts κ, sweeping each model's sparsity parameter. The models are matched on parameter count (section 4.2). Across the swept range every VAEE frontier lies below every SAE frontier: at any given concept count it reconstructs more faithfully, matching the best SAE's reconstruction using roughly an order of magnitude fewer active concepts, and at the dense end of its sweep it reaches a fidelity no SAE attains, even at $k = 128$. A few vector-valued concepts carry reconstruction further than many more scalar atoms — the relaxation of the sparsity–reconstruction trade-off the architecture was designed to deliver.

This advantage is specific to the concept axis. Figure 4.2 repeats the comparison in active *neurons* ℓ_0 — the decoder-capacity unit (sec-

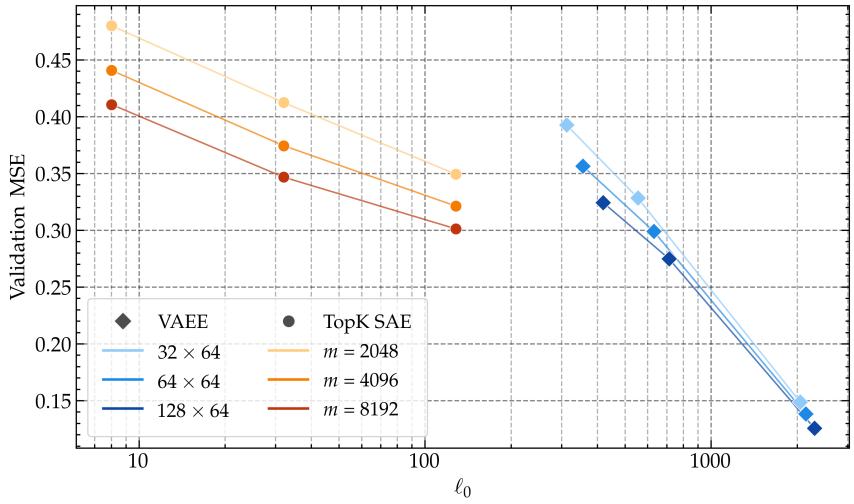

Figure 4.2: The sparsity–reconstruction frontier in active neurons ℓ_0, the decoder-capacity unit, over the same sweeps as fig. 4.1 — the SAE over $k \in \{8, 32, 128\}$ and the VAEE over $\tilde{\kappa} = \pi K \in \{4, 8, 32\}$. The families occupy disjoint ranges — SAEs below $\ell_0 = 128$, VAEEs above $\ell_0 \approx 300$ — so no matched-ℓ_0 comparison is available; the SAE reaches a given error with fewer active neurons.

tion 4.3) — where the two families do not occupy the same range. Because each active VAEE concept is a vector of width $d = 64$, every VAEE operating point sits above $\ell_0 \approx 300$, while the SAEs, with scalar atoms, never exceed $\ell_0 = 128$; a head-to-head reading at matched ℓ_0 is not available from these runs. The trend is nonetheless clear: the SAE reaches a given reconstruction error with fewer active neurons, and the VAEE attains its low concept count by activating more neurons per sample.

The VAEE's local interpretability may be simpler, as it requires inspecting fewer vector-valued concepts rather than a larger number of scalars. Moreover, since the VAEE's dictionary is smaller than a parameter-matched SAE's, fewer dictionary entries need labelling (128 vs 8 192), potentially easing its global interpretation as well.

4.5.1.2 *Behaviour of the Inference Gate*

At inference the gate is continuous (section 3.3.1.2), yet it behaves in practice as a hard one — which is what makes the active-concept count of section 4.3 well defined. Figure 4.3 shows why: the validation-set distribution of the activation probabilities α_k is sharply bimodal, almost all of its mass falling within 0.01 of 0 or 1, the extremes one to two orders of magnitude more frequent than any intermediate value. The active-concept count is therefore insensitive to the exact cut-off, and $1/2$ is a natural threshold for an activation probability.

A complementary property concerns the embeddings the gate acts upon. The conditional-embedding KL of eq. (3.14) carries a

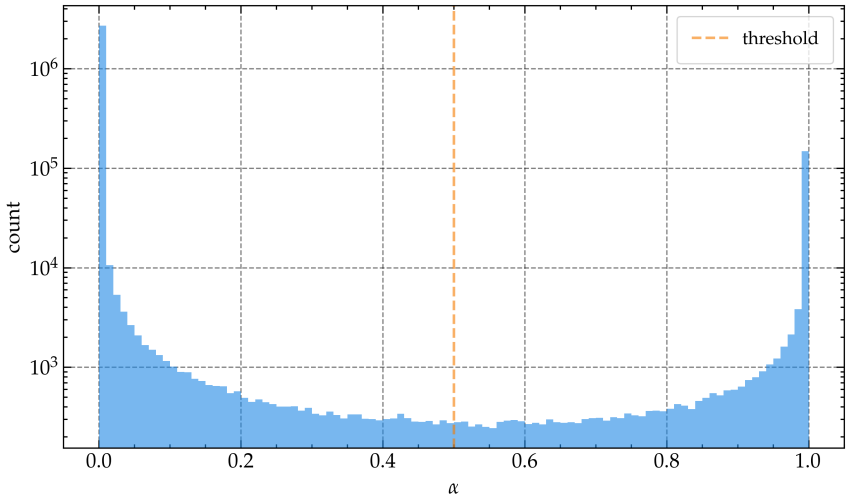

Figure 4.3: Validation-set distribution of the 128×64 VAEE activation probabilities α_k. The mass is sharply bimodal, concentrated within 0.01 of 0 and 1 (each bin spans 0.01), the extreme bins one to two orders of magnitude more frequent than the interior — so the gate is effectively hard at inference and the active-concept count is insensitive to the $1/2$ threshold.

$(1 - \alpha_k)\|\boldsymbol{\mu}_k\|^2$ term that pulls an inactive concept's embedding mean toward the origin, while the multiplicative gate of eq. (3.11) zeroes that slot regardless. The two act on the same inactive concept, which raises the question of whether the penalty does any work on top of the gate. The model design (section 3.3.1.2) assumes it does — that an inactive embedding is held near the origin in any case, the gate only removing the reconstruction gradient that would otherwise inflate it *before* the slot is zeroed.

Figure 4.4 confirms this, though only in part. For every concept and validation token it records the norm of the ungated mean $\|\boldsymbol{\mu}_k(\boldsymbol{x})\|$ — the embedding the penalty regularises, read *before* the gate is applied — split by whether the concept is active ($\alpha_k \geq 1/2$). The two distributions separate clearly. Inactive embeddings are pulled down toward the noise floor: their median norm is 0.96, against 3.62 for active ones — 3.78 times smaller — and 21% fall below $\sigma_0\sqrt{d} = 0.8$, the scale of the noise the posterior itself injects when sampling $\boldsymbol{z}_k = \boldsymbol{\mu}_k + \sigma_0\boldsymbol{\epsilon}$. Active embeddings sit far out, reaching past the prototype-norm scale (median $\|\boldsymbol{h}_k\| = 2.89$) as the same penalty pulls them toward their prototype.

The conditional-embedding KL is therefore doing real work — inactive embeddings end up markedly smaller than active ones — but it does not drive them to zero: a typical inactive embedding still carries a norm near 1, and an average token leaves over a hundred of the 128 concepts inactive, so left ungated their contributions would be far from negligible in aggregate. The soft penalty and the hard gate are thus complementary rather than redundant: the penalty shrinks the

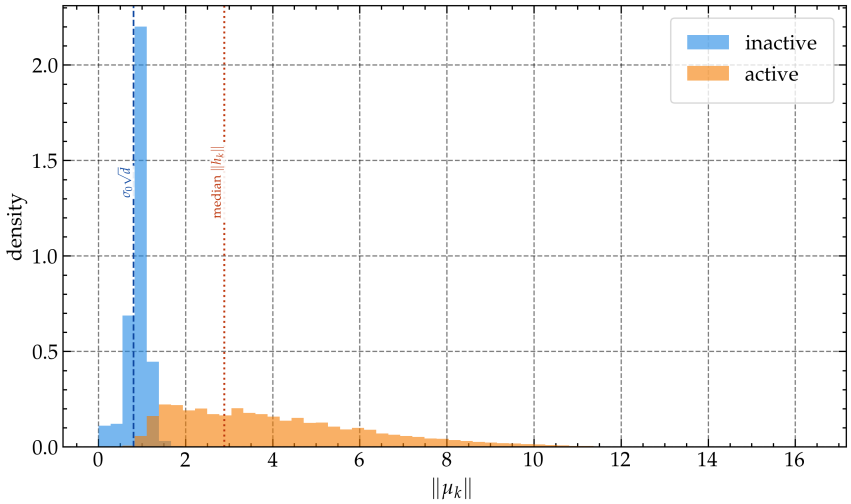

Figure 4.4: Validation-set distribution of the ungated embedding-mean norm $\|\boldsymbol{\mu}_k\|$ for the 128×64 VAEE, over all concept–token pairs, split into active ($\alpha_k \geq 1/2$) and inactive. Inactive embeddings are pulled down near the sampling noise floor $\sigma_0\sqrt{d} = 0.8$ (dashed), their median norm (0.96) some 3.78 times smaller than that of active ones, which spread out past the median prototype norm $\|\boldsymbol{h}_k\| = 2.89$ (dotted). The conditional-embedding KL shrinks the inactive embeddings *before* the gate acts but does not zero them, so penalty and gate suppress inactive concepts together.

inactive embeddings, and the gate removes what remains — which is also what makes the mask a clean handle for the interventions of section 3.3.3.

4.5.2 QUALITATIVE RESULTS

The frontier of section 4.5.1.1 measures how much reconstruction each active concept accounts for, but not what those concepts are. A unit is useful for interpretability only if it is human-comprehensible, so this section reads off each model's *concept dictionary*: for every concept, a collection of tokens on which it activates most strongly, each shown inside the context it was drawn from with the activating token in bold [2, 4]. A concept is comprehensible when those tokens share a discernible property — a meaning, a grammatical class, an orthographic pattern — and opaque when they do not.

A single operating point is inspected per model — the parameter-matched pair of the 128×64 VAEE at $\tilde{\kappa} = 8$ and the $m = 8\,192$ TopK SAE at $k = 8$ — as concept quality was not measured along the frontier.

THE VAEE CONCEPT DICTIONARY Table 4.3 shows three of its concepts. Some are genuinely semantic: concept C6 fires on a vocabulary of dread (`curse`, `tension`, `fear`, `creepy`); C48 on modifiers of mag-

C6 — fear / horror / suspense	(firing rate 10.2%)
`... of nuclear holoca**ust** to generate cheap hol ...`	
`... superstitious **curse** on chain letters and ...`	
`... cheap hollywood **tension** .`	
`... its our substantial collective **fear** of nuclear holoca ...`	
`an effectively **creep**y , fear- ...`	
`... effectively creepy , **fear**-inducing ...`	
`... lrb- not **fear**-reducing - ...`	
C48 — magnitude / intensity modifiers	(firing rate 9.1%)
`... video game is a **lot** more fun than the ...`	
`... 9 exploits our **substantial** collective fear of nuclear ...`	
`... game is a lot **more** fun than the film ...`	
`... yrical work of **considerable** force and truth .`	
`... it 's a **stunning** lyrical work ...`	
`... , and feelings that **profound**ly deepen them ...`	
C123 — word-final subword fragments	(firing rate 15.9%)
`... -trip dre**k** we 've seen ...`	
`... enough for sham**u** the killer whale ...`	
`... fully amusing rom**p** that , if nothing ...`	
`... of naivet**é** , passion and talent ...`	
`... film seems thirst**y** for reflection , itself ...`	
`... fancy a real down**er** ?`	

Table 4.3: Three concepts from the $\tilde{\kappa} = 8$, 128×64 VAEE dictionary, each shown with a few of its strongest activations and the activating token in bold. C6 and C48 are semantically coherent; C123 fires on the final sub-token of a word regardless of meaning. The firing rate is the fraction of validation tokens on which the concept is active.

nitude (`substantial`, `considerable`, `stunning`, `broad`); C123 fires on the final sub-token of a word irrespective of meaning (the `k` of `drek`, the `p` of `romp`, the `é` of `naiveté`) — an orthographic regularity rather than a semantic concept. The VAEE dictionary is a mixture: a minority of legible concepts among a majority of uninterpretable units that fire on seemingly random tokens.

THE SAE CONCEPT DICTIONARY The SAE dictionary (table 4.4) is similar in kind. It too has clean entries — feature C6703 on nouns of film and storytelling (`plot`, `journey`, `movie`, `performance`) and C958 on possessive determiners (`my`, `your`, `our`) — among a number of incoherent ones. A characteristic failure is *redundancy*: groups of features collapse onto seemingly random, near-identical token sets, three of which are shown in the lower block of table 4.4. This feature-splitting is a known tendency of over-complete dictionaries [4, 41], and an

C6703 — film and storytelling nouns	(firing rate 1.4%)
`... -channel kind of **plot** and a lead actress ...`	
`... a much more emotional **journey** than what shyam ...`	
`... cookie-cutter **movie** , a cut- ...`	
`... he is in this **performance** .`	
`... one of those baseball **pictures** where the hero is ...`	
`... lovely film with lovely **performances** by buy and acc ...`	
C958 — possessive determiners	(firing rate 0.5%)
`**my** wife is an actress ...`	
`the lower **your** expectations , the more ...`	
`... 't aim for **our** sympathy , but rather ...`	
C6649, **C2491**, **C5296** — three near-duplicate features	(firing rate 3.8% each)
`C6649: deep, leave, coming, six, instead, series, board, ...`	
`C2491: deep, leave, coming, six, instead, series, board, ...`	
`C5296: deep, leave, coming, six, instead, series, board, ...`	

Table 4.4: Features from the $k = 8$, $m = 8\,192$ TopK SAE dictionary. C6703 and C958 are coherent; the lower block lists the highest-activating tokens of three features that converge on a near-identical set — one instance of the redundancy that consumes a large share of the dictionary. Firing rate as in table 4.3.

$m = 8\,192$ dictionary trained on such a small corpus has ample room for it.

NEITHER DICTIONARY IS CLEARLY MORE INTERPRETABLE Read side by side, the two are about equally comprehensible. Each yields a handful of coherent units and a majority that don't have a clear label, and the per-concept reconstruction advantage of section 4.5.1.1 does not show up as visibly cleaner concepts. Notably, the comparison is not perfectly even: the VAEE's meaningful concepts come from a dictionary of 128, the SAE's from one of 8 192, so a comparable number of usable units is recovered from far fewer of them. On this evidence the VAEE neither clearly improves nor clearly degrades interpretability relative to the SAE. A larger, random sample of both dictionaries is reproduced in appendix B.

Whether the VAEE's vector concepts convey richer interpretability than scalar features is therefore left open — a question for a larger and more varied corpus.

4.5.3 ABLATION STUDY

The configuration used in the experiments above was not the only one available. It fixes an embedding width of $d = 64$ and a linear-plus-

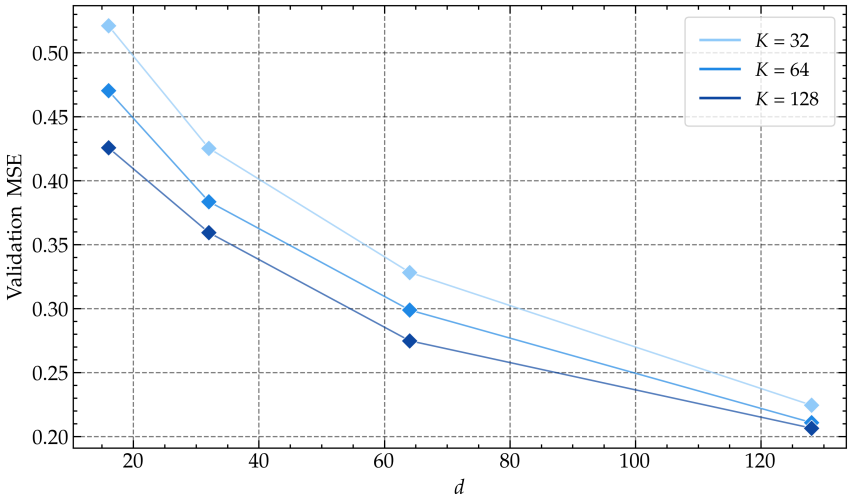

Figure 4.5: Effect of the per-concept embedding width d on VAEE reconstruction. Validation MSE against $d \in \{16, 32, 64, 128\}$, one curve per dictionary size $K \in \{32, 64, 128\}$, all at the fixed sparsity target $\tilde{\kappa} = \pi K = 8$, so that only the embedding width varies along each curve. Reconstruction improves with d, with diminishing returns — widening a concept adds capacity that a scalar atom cannot, though at the cost of more active neurons ($\ell_0 = \kappa d$) and more parameters.

GELU encoder with a linear decoder. This section reports the ablations behind those choices.

EMBEDDING WIDTH The embedding width d is the VAEE's defining degree of freedom — the dimension along which a concept becomes a vector rather than a scalar. Figure 4.5 sweeps it, plotting validation MSE against $d \in \{16, 32, 64, 128\}$ at a fixed $\tilde{\kappa} = \pi K$ for three dictionary sizes. Reconstruction improves monotonically with d, with diminishing returns: the gain from 16 to 64 is larger than from 64 to 128. This extra capacity is what a scalar atom cannot supply, but it comes at a price: every active concept now contributes d active neurons rather than one ($\ell_0 = \kappa d$, section 4.3). The $d = 64$ used in the other experiments already captures much of the reconstruction gain while keeping training feasible on limited resources, since increasing d with all other dimensions fixed raises the parameter count in proportion — the encoder and decoder weights scale as $2nKd$ (section 4.2).

ENCODER AND DECODER DEPTH Throughout, the VAEE uses a *shallow* configuration: a single linear-plus-GELU layer for the encoder f_ϕ and a purely linear decoder g_θ. Two alternatives were explored: an MLP for either map, and a plain linear encoder and decoder. Figure 4.6 compares them at a fixed dictionary size and embedding width. The added depth does not translate into better performance: the shallow configuration reconstructs better, and at a lower active-concept

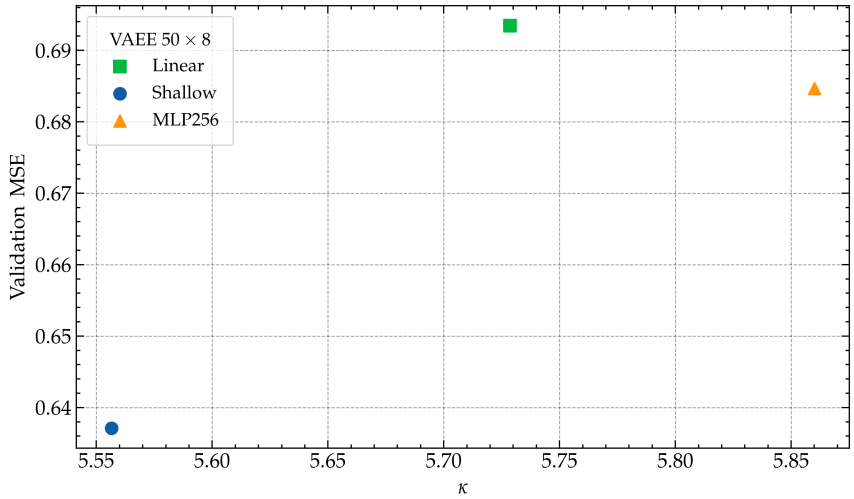

Figure 4.6: Validation MSE of the VAEE under encoder and decoder architectures of increasing depth — a plain linear encoder, the shallow linear-plus-GELU encoder used throughout, and an MLP with a hidden layer 256 neurons wide — at a fixed dictionary size $K = 50$ and embedding width $d = 8$. The shallow configuration achieves better reconstruction at a lower κ than both the linear and MLP configurations.

count, than either alternative. The MLP's larger parameter count does not significantly improve reconstruction over the plain-linear configuration either, despite activating more concepts.

DESIGN CHOICES LEFT OPEN Two parts of the setup were fixed rather than ablated. First, the gate measures embedding-to-prototype agreement by cosine similarity rather than the inner-product or negative-Euclidean alternatives of eq. (3.12); cosine is a reasonable default — it decouples the gate from the embedding magnitude (section 3.3.1.3) — but the alternatives were not swept. Second, the embedding-posterior standard deviation σ_0 was held fixed throughout. How either choice affects reconstruction and interpretability is left to future work (section 5.2).

CONCLUSIONS

This chapter tested the VAEE's central claim on the decomposition of LLM activations, against a parameter-matched TopK SAE on the residual stream of Mistral 7B (section 4.1). On the axis that matters for interpretability — the number of active concepts κ — the result is clear: every VAEE frontier lies below every SAE frontier, matching the best SAE's reconstruction with roughly an order of magnitude fewer active concepts (section 4.5.1.1). The relaxation of the sparsity–reconstruction trade-off the architecture was designed for does show up, but on this

axis alone: measured in active *neurons* ℓ_0, the trade-off is not removed but relocated, the SAE reaching a given error with fewer active scalars.

The interpretability of the recovered concepts is a more equivocal story. Read side by side, neither dictionary is clearly more interpretable than the other (section 4.5.2): each yields a handful of coherent units among a majority that lack a clear interpretation, and the per-concept reconstruction advantage does not translate into visibly cleaner concepts. Both likely reflect the limits of the data: the small and thematically narrow SST-2 offers too little variety for a rich concept vocabulary to form. The ablations (section 4.5.3) identify the embedding width d as the architecture's decisive degree of freedom and show that the shallow configuration outperforms all explored variants. Whether vector concepts afford richer interpretability than scalar features is a question a larger corpus and a dedicated evaluation must settle — taken up as future work in section 5.2.

5 CONCLUSIONS

Concept-based methods describe a model's computation in human-meaningful terms, but they encode each concept as a single scalar — a unit in the bottleneck of a CBM, or an atom in the dictionary of an SAE — and so trade representational capacity for simplicity. This thesis set out to lift that restriction. It introduced the VAEE (section 3.3), a model that represents each concept as a learned vector embedding rather than a scalar, and demonstrated its viability on the decomposition of LLM activations. This chapter draws together what the work contributes (section 5.1) and the directions it leaves open (section 5.2).

5.1 CONTRIBUTIONS

The central contribution is the VAEE itself. The encoder maps an input to one vector embedding per concept; a learned prototype for each concept then decides, by similarity, which concepts are active; and the input is reconstructed from the stack of those active embeddings. Giving each concept a vector rather than a scalar recovers the representational capacity a scalar bottleneck gives up, carrying the vector-valued concepts of CEMs into the unsupervised setting of activation decomposition and combining them with the variational inference and discrete latents of VAEs and VQ-VAEs. Rather than assemble these ingredients by analogy, chapter 3 derives the model from first principles: a generative model with a gated-Gaussian prior, an ELBO that separates reconstruction from a mask prior and a conditional-embedding term, and a gating rule that follows as the optimal mask posterior.

The model's capabilities were demonstrated on one application, the decomposition of an LLM's residual-stream activations. Against a parameter-matched TopK SAE, the VAEE matches the reconstruction error using roughly an order of magnitude fewer active concepts (section 4.5.1.1) — evidence that vector-valued concepts carry more information per unit of sparsity. A qualitative comparison of the recovered concept dictionaries (section 4.5.2) and a set of ablations over embedding width and encoder–decoder depth (section 4.5.3) round out the picture of how the architecture behaves.

The evidence shown is, admittedly, limited. Reconstruction was assessed only by mean squared error; each VAEE concept now spans several active neurons rather than one, so fewer active *concepts* need not mean fewer active *neurons*; and the recovered concepts were not found

more interpretable than the SAE's. The contribution, then, is the added representational capacity of vector-valued concepts and the principled model that delivers it — not a claim about the interpretability of what it recovers.

5.2 FUTURE WORK

The clearest open question is interpretability. The VAEE matches the SAE on reconstruction with far fewer active concepts, but its dictionaries were not found to be more comprehensible, and both were limited by the small evaluation corpus (section 4.5.2). Whether vector-valued concepts are more interpretable than scalar ones is therefore left open, and answering it would need a larger and more varied dataset. Moreover, additional tests of reconstruction quality beyond mean squared error should be performed, such as those standard in SAE evaluation (cross-entropy or perplexity of the spliced model), which this thesis did not run for the comparison.

A second direction is the use of the recovered concepts to steer model behaviour. Section 3.3.3 formulates concept removal and insertion directly from the gated mask, with single-concept edits that stay in distribution, but the thesis stops at the formulation: the effect of such interventions on an LLM's output was not measured, and quantifying it is a natural next step.

Finally, activation decomposition is only one use of the architecture. The VAEE is a general representation-learning model, and how its vector-valued concepts compare with the latents of VAEs and VQ-VAEs on disentanglement and downstream tasks remains to be explored.

A

CLOSED-FORM CONDITIONAL KL

The conditional-embedding KL of eq. (3.13) reduces to the closed form substituted into the practical loss (eq. (3.14)). The expectation over the Bernoulli mask $\mathbb{E}_{q_\phi(c_k|\boldsymbol{x})}$ is the convex combination of its two branches with weights α_k and $1-\alpha_k$:

$$\sum_{k=1}^{K} \mathbb{E}_{q_\phi(c_k|\boldsymbol{x})}\left[D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_k \mid c_k)\right)\right] = \\ = \sum_{k=1}^{K} \left[\alpha_k \, D_{\mathrm{KL}}^{(k,1)} + (1-\alpha_k)\, D_{\mathrm{KL}}^{(k,0)}\right],$$

where $D_{\mathrm{KL}}^{(k,c)} = D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_k \mid c_k{=}c)\right)$. Both branches are KL divergences between Gaussians sharing the covariance $\sigma_0^2\mathbb{I}$, for which the divergence reduces to the scaled squared distance between the means,

$$D_{\mathrm{KL}}\left(\mathcal{N}(\boldsymbol{\mu}, \sigma_0^2\mathbb{I}) \,\|\, \mathcal{N}(\boldsymbol{\nu}, \sigma_0^2\mathbb{I})\right) = \frac{1}{2\sigma_0^2}\|\boldsymbol{\mu}-\boldsymbol{\nu}\|^2.$$

With the prior means $c_k\boldsymbol{h}_k$ of eq. (3.3) — the prototype $\boldsymbol{h}_k$ when active, the origin when inactive — the conditional-embedding KL takes its closed form,

$$\sum_{k=1}^{K} \mathbb{E}_{q_\phi(c_k|\boldsymbol{x})}\left[D_{\mathrm{KL}}\left(q_\phi(\boldsymbol{z}_k \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z}_k \mid c_k)\right)\right] = \\ = \frac{1}{2\sigma_0^2}\sum_{k=1}^{K}\left[\alpha_k\|\boldsymbol{\mu}_k - \boldsymbol{h}_k\|^2 + (1-\alpha_k)\|\boldsymbol{\mu}_k\|^2\right], \quad \text{(3.14 revisited)}$$

which is the term substituted into the practical loss of eq. (3.15), with the factor $\frac{1}{2\sigma_0^2}$ being absorbed into the γ hyperparameter.

B

CONCEPT DICTIONARIES

The three concepts shown per model in section 4.5.2 are curated to illustrate particular kinds of concepts. This appendix instead reproduces a *random*, uncurated sample of concepts from each of the same two models — the $\tilde{\kappa} = 8$, 128×64 VAEE and the $k = 8$, $m = 8\,192$ TopK SAE — so that the population claims of section 4.5.2 can be checked directly rather than taken on the strength of a hand-picked few. Each concept is shown with a few of its strongest activations and the activating token in bold; the firing rate is the fraction of validation tokens on which the concept is active.

VAEE CONCEPT DICTIONARY

C3 (firing rate 10.0%)

```
... generate cheap holly wood tension .
... superstitious curse on chain letters and ...
... cheap hollywood tension .
... substantial collective fear of nuclear holocaust to ...
... its our substantial collective fear of nuclear holoca ...
```

C9 (firing rate 9.1%)

```
harrison 's flowers puts its ...
k-19 explo ...
k-19 exploits ...
one long string of cl ...
k-19 exploits our ...
```

C18 (firing rate 9.5%)

```
... ust to generate cheap holl ywood tension .
... ering clear of knee - jerk reactions and ...
... steering clear of knee -jerk reactions ...
sel dom has a movie ...
... funny , but down right repellent ...
```

C21 (firing rate 10.4%)

```
... istic performance speaks volumes more truth than any ' ...
an effectively creepy , fear ...
... an effectively creepy , fear - induc ...
... s played in the most straight - faced fashion ...
```

... -lrb- **not** fear-reducing ...

C25 (firing rate 10.2%)

... comedy about all there **is** to love - and ...
... strangers actually shows may **put** you off the idea ...
... generate cheap holly**wood** tension .
... chain letters and actually **applies** it .
... hudlin can **make** it more than fit ...

C26 (firing rate 9.8%)

it **'**s played in the ...
like leon , it ...
... get his shot at **the** big time .
... his shot at the **big** time .
... **'** attempts to get **his** shot at the big ...

C29 (firing rate 9.2%)

... -19 explo**its** our substantial collective fear ...
... annabe comic ad**ams** **'** attempts to get ...
... about a guy who **is** utterly unlikeable , ...
... s three lead actress**es** .
it **'s** like every bad idea ...

C31 (firing rate 9.3%)

... fanciful touches **,** the film is a ...
... **'** entertaining film chron**icles** seinfeld **'**s ...
... feld **'**s return **to** stand-up comedy ...
char**les** **'** entertaining film chron ...
... ific performances from them **all** .

C33 (firing rate 9.6%)

... iche so lacking in **original**ity that if you ...
... if you stripped away **its** inspirations there would ...
... ile long on am**iable** monkeys and worthy ...
... keys and worthy environmental**ism** , jane good ...
... monkeys and worthy **environmental**ism , jane ...

C49 (firing rate 10.0%)

... for horror movie fan**at**ics .
... ese director hideo **n**akata , who ...
... japanese director **h**ideo nakata ...
... in grand , un**com**plicated fashion .
... in an unn**erv**ing way .

C71 (firing rate 10.9%)

it **'**s played in the ...
... the superstit**ious** curse on chain letters ...

... film from japan**ese** director hideo n ...

... ocaust to generate **cheap** hollywood tension ...

an effectively creep**y** , fear-ind ...

C72 (firing rate 9.6%)

... s heart and following **the** demands of tradition .

... n't smell **this** turkey rotting ...

involves **two** mysteries - one ...

... british court **'s** extradition che ...

... and the regime **'s** talking-head survivors ...

C75 (firing rate 9.2%)

it 's **a** beautiful madness .

one long string of cl ...

partway through watching this ...

you really have to wonder ...

part**way** through watching this sac ...

C76 (firing rate 13.5%)

... long string of cl**iches** .

... ever entertained the **notion** of doing what the ...

... heartstring untug**ged** and no liberal cause ...

... cause unplund**ered** .

... vs. them " **opera** that leaves no heart ...

C81 (firing rate 13.5%)

... k-19 **explo**its our substantial collective ...

... who takes the super**st**itious curse on ...

... collective fear of nuclear **hol**ocaust to generate ...

... takes the superst**it**ious curse on chain ...

... fear of nuclear hol**oca**ust to generate cheap ...

C83 (firing rate 9.5%)

a **tender** , witty , ...

... wannabe comic **ad**ams ' attempts to ...

... his shot at the **big** time .

... itious curse on **chain** letters and actually applies ...

my big fat **g**reek wedding uses stere ...

C88 (firing rate 11.1%)

... attempts to get his **shot** at the big time ...

... akata , who **takes** the superstit ...

... of nuclear holoca**ust** to generate cheap hol ...

... its our substantial collective **fear** of nuclear holoca ...

... letters and actually applies **it** .

C97 (firing rate 9.1%)

... hollywood tension **.**

... to get his shot **at** the big time .

... get his shot at **the** big time .

... i found myself confused **when** it came time to ...

... get to the heart **of** the movie .

C98 (firing rate 9.9%)

... 19 exploits **our** substantial collective fear of ...

... the superstit**ious** curse on chain letters ...

... our substantial collective fear **of** nuclear holocaust ...

... 've ever entert**ained** the notion of doing ...

one long string **of** cliches .

C101 (firing rate 11.0%)

... laced with humor **and** a few fanc ...

... chain letters and actually **applies** it .

... ata , who takes **the** superstitious ...

... aced with humor and **a** few fanciful ...

... astounding given **the** production 's a ...

SAE CONCEPT DICTIONARY

C68 (firing rate 3.8%)

<bos> one long string of ...

... feelings that profoundly **deep**en them .

... much more when you **leave** .

the second **coming** of harry pot ...

... ford movie - **six** days , seven nights ...

C678 (firing rate 0.4%)

... -lrb- **grant** 's -rr ...

it 's played in ...

... lrb- grant **'**s -rrb ...

... s -rrb**-** bumbling magic takes ...

minority report is exactly what ...

C1006 (firing rate 0.4%)

... thought they could achieve **an** air of frantic ...

... seems to be under **the** illusion that he ' ...

... umes and sets are **grand** .

... teary-eyed **original** .

... worthy of pucc**ini** than they are of ...

C1514 (firing rate 3.8%)

<bos> one long string of ...
... feelings that profoundly **deep**en them .
... much more when you **leave** .
the second **coming** of harry pot ...
... ford movie - **six** days , seven nights ...

C2082 (firing rate 3.8%)

<bos> one long string of ...
... feelings that profoundly **deep**en them .
... much more when you **leave** .
the second **coming** of harry pot ...
... ford movie - **six** days , seven nights ...

C2115 (firing rate 3.8%)

<bos> one long string of ...
... feelings that profoundly **deep**en them .
... much more when you **leave** .
the second **coming** of harry pot ...
... ford movie - **six** days , seven nights ...

C2840 (firing rate 0.2%)

... with resonance by**<bos>** ilya chaiken ...
... mull over in **terms** of love , loyalty ...
... can relate to the **search** for inner peace by ...
... in the first few **minutes** .
... umes and sets are **grand** .

C3393 (firing rate 0.5%)

the jabs it em ...
... fiercely at**he**istic hero .
... a rosily **my**opic view of life ...
... make a playground **out** of the refuse of ...
... it remains depressingly **pro**saic and dull ...

C3716 (firing rate 0.5%)

... lighthearted , **it** offends by just ...
... xenophob**ic** pedagogy that ...
... ovie franchise game , **i** spy makes its big ...
... quality of a lesser **h**arrison ford ...
... opening scenes , it 's clear that all ...

C3915 (firing rate 0.6%)

... sporting a bree**zy** spontaneity and ...
... s effort has tons **of** charm and the wh ...
... ity and realistically **drawn** characterizations , develop ...

... of the leads keep **the** film grounded and ...
... film , sporting **a** breezy spontane ...

C4164 (firing rate 3.9%)

... ahhhh ... **revenge** is sweet !
... hhhh ... revenge **is** sweet !
... superstitious **curse** on chain letters and ...
... this film , unlike **other** dumas adapt ...
... likened to a **treasure** than a lengthy ...

C4173 (firing rate 0.9%)

... operatic grandeur **.**
... ‘ picture shows . ’
... status as a film **maker** who artfully b ...
... b- hile **long** on amiable mon ...
... richness of its **performances** .

C4526 (firing rate 0.7%)

... hidden it from everyone **.**
... loser is the **audience** .
... ious to be engaging **...** the isle def ...
... an actual feature movie **,** the kind that charges ...
... tell it to us **?**

C4527 (firing rate 0.4%)

... ful , intelligent film **that** stays within the conf ...
... intriguing documentary **which** is emotionally dilut ...
... turn thriller that **certainly** should n’t ...
... dom has a movie **so** closely matched the spirit ...
... as a film maker **who** artfully bends ...

C4551 (firing rate 0.4%)

confirms the nag ...
a very well-made ...
scores no points for original ...
people cinema at its finest ...
part low rent godfather ...

C4679 (firing rate 0.4%)

a very well-made ...
... can relate to the **search** for inner peace by ...
... in the first few **minutes** .
... umes and sets are **grand** .
... teary-eyed **original** .

C5613 (firing rate 0.5%)

... adillo ’s **dark** and jolting ...

the jabs it em ...

... johnny d**ank**worth 's best ...

... can relate to the **search** for inner peace by ...

... umes and sets are **grand** .

C5811 (firing rate 0.7%)

... is n't **any** funnier than bad ...

... n't funny **some** of the time - ...

... is n't **even** all that dumb .

... n't even **all** that dumb .

... do n't **enable** them to discern ...

C6249 (firing rate 0.3%)

it '**s** played in the most ...

man**ages** to transcend the ...

in **the** end , we are ...

... in the first few **minutes** .

... umes and sets are **grand** .

C6331 (firing rate 0.4%)

it 's played in ...

... feelings that profoundly **deep**en them .

... much more when you **leave** .

the second **coming** of harry pot ...

... ford movie - **six** days , seven nights ...

BIBLIOGRAPHY

[1] Emmanuel Ameisen et al. *Circuit Tracing: Revealing Computational Graphs in Language Models*. Tech. rep. Transformer Circuits Thread, Mar. 2025. URL: https://transformer-circuits.pub/2025/attribution-graphs/methods.html (visited on 11/06/2026).

[2] Steven Bills, Nick Cammarata, Dan Mossing, Henk Tillman, Leo Gao, Gabriel Goh, Ilya Sutskever, Jan Leike, Jeff Wu and William Saunders. *Language Models Can Explain Neurons in Language Models*. Tech. rep. OpenAI, May 2023. URL: https://openaipublic.blob.core.windows.net/neuron-explainer/paper/index.html (visited on 11/06/2026).

[3] David M. Blei, Alp Kucukelbir and Jon D. McAuliffe. 'Variational Inference: A Review for Statisticians'. In: *Journal of the American Statistical Association* 112.518 (Apr. 2017), pp. 859–877. ISSN: 0162-1459. DOI: 10.1080/01621459.2017.1285773. URL: https://doi.org/10.1080/01621459.2017.1285773 (visited on 17/06/2026).

[4] Trenton Bricken et al. *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*. Tech. rep. Transformer Circuits Thread, Oct. 2023. URL: https://transformer-circuits.pub/2023/monosemantic-features/index.html (visited on 11/06/2026).

[5] Tom Brown et al. 'Language Models Are Few-Shot Learners'. In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. URL: https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfcb4967418bfb8ac142f64a-Abstract.html (visited on 13/06/2026).

[6] Francesco De Santis, Philippe Bich, Gabriele Ciravegna, Pietro Barbiero, Tania Cerquitelli and Danilo Giordano. 'Towards Better Generalization and Interpretability in Unsupervised Concept-Based Models'. In: *Machine Learning and Knowledge Discovery in Databases. Research Track: European Conference, ECML PKDD 2025, Porto, Portugal, September 15–19, 2025, Proceedings, Part III*. Berlin, Heidelberg: Springer-Verlag, Oct. 2025, pp. 478–494. ISBN: 978-3-032-06065-5. DOI: 10.1007/978-3-032-06066-2_28. URL: https://doi.org/10.1007/978-3-032-06066-2_28 (visited on 11/06/2026).

[7] Qingxiu Dong et al. 'A Survey on In-context Learning'. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Ed. by Yaser Al-Onaizan, Mohit Bansal and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 1107–1128. DOI: 10.18653/v1/2024.emnlp-main.64. URL: https://aclanthology.org/2024.emnlp-main.64/ (visited on 11/06/2026).

[8] Jacob Dunefsky, Philippe Chlenski and Neel Nanda. 'Transcoders Find Interpretable LLM Feature Circuits'. In: *Advances in Neural Information Processing Systems* 37 (Dec. 2024), pp. 24375–24410. DOI: 10.52202/079017-0768. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/2b8f4db0464cc5b6e9d5e6bea4b9f308-Abstract-Conference.html (visited on 10/06/2026).

[9] Nelson Elhage et al. *A Mathematical Framework for Transformer Circuits*. Tech. rep. Transformer Circuits Thread, Dec. 2021. URL: https://transformer-circuits.pub/2021/framework/index.html (visited on 13/06/2026).

[10] Nelson Elhage et al. *Toy Models of Superposition*. Tech. rep. Transformer Circuits Thread, Sept. 2022. URL: https://transformer-circuits.pub/2022/toy_model/index.html (visited on 11/06/2026).

[11] Mateo Espinosa Zarlenga et al. 'Concept Embedding Models: Beyond the Accuracy-Explainability Trade-Off'. In: *Advances in Neural Information Processing Systems* 35 (Dec. 2022), pp. 21400–21413. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/867c06823281e506e8059f5c13a57f75-Abstract-Conference.html (visited on 12/06/2026).

[12] Leo Gao, Tom Dupre la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike and Jeffrey Wu. 'Scaling and Evaluating Sparse Autoencoders'. In: *The Thirteenth International Conference on Learning Representations*. Oct. 2024. URL: https://openreview.net/forum?id=tcsZt9ZNKD (visited on 14/05/2026).

[13] Ian Goodfellow, Aaron Courville and Yoshua Bengio. *Deep Learning*. Adaptive Computation and Machine Learning. Cambridge, Massachusetts: The MIT Press, 2016. ISBN: 978-0-262-03561-3 978-0-262-33737-3.

[14] Irina Higgins, Loic Matthey, Arka Pal, Christopher Burgess, Xavier Glorot, Matthew Botvinick, Shakir Mohamed and Alexander Lerchner. 'Beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework'. In: *International Conference on Learning Representations*. Feb. 2017. URL: https://openreview.net/forum?id=Sy2fzU9gl (visited on 10/06/2026).

[15] Robert Huben, Hoagy Cunningham, Logan Riggs Smith, Aidan Ewart and Lee Sharkey. 'Sparse Autoencoders Find Highly Interpretable Features in Language Models'. In: *The Twelfth International Conference on Learning Representations*. Oct. 2023. URL: https://openreview.net/forum?id=F76bwRSLeK (visited on 28/05/2026).

[16] Eric Jang, Shixiang Gu and Ben Poole. 'Categorical Reparameterization with Gumbel-Softmax'. In: *International Conference on Learning Representations*. Feb. 2017. URL: https://openreview.net/forum?id=rkE3y85ee (visited on 14/05/2026).

[17] Albert Q. Jiang et al. *Mistral 7B*. Oct. 2023. DOI: 10.48550/arXiv.2310.06825. arXiv: 2310.06825 [cs.CL]. URL: http://arxiv.org/abs/2310.06825 (visited on 18/06/2026).

[18] Been Kim, Rajiv Khanna and Oluwasanmi O Koyejo. 'Examples Are Not Enough, Learn to Criticize! Criticism for Interpretability'. In: *Advances in Neural Information Processing Systems*. Vol. 29. Curran Associates, Inc., 2016. URL: https://proceedings.neurips.cc/paper_files/paper/2016/hash/5680522b8e2bb01943234bce7bf84534-Abstract.html (visited on 11/06/2026).

[19] Been Kim, Martin Wattenberg, Justin Gilmer, Carrie Cai, James Wexler, Fernanda Viegas and Rory Sayres. 'Interpretability Beyond Feature Attribution: Quantitative Testing with Concept Activation Vectors (TCAV)'. In: *Proceedings of the 35th International Conference on Machine Learning*. PMLR, July 2018, pp. 2668–2677. URL: https://proceedings.mlr.press/v80/kim18d.html (visited on 11/06/2026).

[20] Diederik P. Kingma and Jimmy Ba. *Adam: A Method for Stochastic Optimization*. Jan. 2017. DOI: 10.48550/arXiv.1412.6980. arXiv: 1412.6980 [cs.LG]. URL: http://arxiv.org/abs/1412.6980 (visited on 19/06/2026).

[21] Diederik P. Kingma and Max Welling. 'Auto-Encoding Variational Bayes'. In: International Conference on Learning Representations, May 2014. DOI: 10.48550/arXiv.1312.6114. arXiv: 1312.6114 [stat]. URL: http://arxiv.org/abs/1312.6114 (visited on 07/11/2025).

[22] Pang Wei Koh, Thao Nguyen, Yew Siang Tang, Stephen Mussmann, Emma Pierson, Been Kim and Percy Liang. 'Concept Bottleneck Models'. In: *Proceedings of the 37th International Conference on Machine Learning*. PMLR, Nov. 2020, pp. 5338–5348. URL: https://proceedings.mlr.press/v119/koh20a.html (visited on 11/06/2026).

[23] Scott M Lundberg and Su-In Lee. 'A Unified Approach to Interpreting Model Predictions'. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html (visited on 01/06/2026).

[24] Chris J. Maddison, Andriy Mnih and Yee Whye Teh. 'The Concrete Distribution: A Continuous Relaxation of Discrete Random Variables'. In: *International Conference on Learning Representations*. Feb. 2017. URL: https://openreview.net/forum?id=S1jE5L5gl (visited on 14/05/2026).

[25] Samuel Marks, Can Rager, Eric Michaud, Yonatan Belinkov, David Bau and Aaron Mueller. 'Sparse Feature Circuits: Discovering and Editing Interpretable Causal Graphs in Language Models'. In: *International Conference on Learning Representations* 2025 (May 2025), pp. 23888–23923. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/3ba4d47a83e498c2b1a0868cba20f6de-Abstract-Conference.html (visited on 10/06/2026).

[26] Christoph Molnar. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. 3rd edition. Munich, Germany: Christoph Molnar, 2025. ISBN: 978-3-911578-03-5.

[27] Tuomas Oikarinen, Subhro Das, Lam M. Nguyen and Tsui-Wei Weng. 'Label-Free Concept Bottleneck Models'. In: *The Eleventh International Conference on Learning Representations*. Sept. 2022. URL: https://openreview.net/forum?id=FlCg47MNvBA (visited on 11/06/2026).

[28] Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov and Shan Carter. 'Zoom In: An Introduction to Circuits'. In: *Distill* 5.3 (Mar. 2020), e00024.001. ISSN: 2476-0757. DOI: 10.23915/distill.00024.001. URL: https://distill.pub/2020/circuits/zoom-in (visited on 04/05/2026).

[29] Bruno A. Olshausen and David J. Field. 'Sparse Coding with an Overcomplete Basis Set: A Strategy Employed by V1?' In: *Vision Research* 37.23 (Dec. 1997), pp. 3311–3325. ISSN: 0042-6989. DOI: 10.1016/S0042-6989(97)00169-7. URL: https://www.sciencedirect.com/science/article/pii/S0042698997001697 (visited on 28/05/2026).

[30] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei and Ilya Sutskever. *Language Models Are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019. (Visited on 13/06/2026).

[31] Senthooran Rajamanoharan, Arthur Conmy, Lewis Smith, Tom Lieberum, Vikrant Varma, János Kramár, Rohin Shah and Neel Nanda. 'Improving Sparse Decomposition of Language Model Activations with Gated Sparse Autoencoders'. In: *Advances in Neural Information Processing Systems* 37 (Dec. 2024), pp. 775–818. DOI: 10.52202/079017-0024. URL: https://proceedings.neurips.cc/paper_files/paper/2024/hash/01772a8b0420baec00c4d59fe2fbace6-Abstract-Conference.html (visited on 05/06/2026).

[32] Senthooran Rajamanoharan, Tom Lieberum, Nicolas Sonnerat, Arthur Conmy, Vikrant Varma, János Kramár and Neel Nanda. *Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders*. Aug. 2024. DOI: 10.48550/arXiv.2407.14435. arXiv: 2407.14435 [cs]. URL: http://arxiv.org/abs/2407.14435 (visited on 23/02/2026).

[33] Danilo Jimenez Rezende, Shakir Mohamed and Daan Wierstra. 'Stochastic Backpropagation and Approximate Inference in Deep Generative Models'. In: *Proceedings of the 31st International Conference on Machine Learning*. PMLR, June 2014, pp. 1278–1286. URL: https://proceedings.mlr.press/v32/rezende14.html (visited on 10/06/2026).

[34] Marco Tulio Ribeiro, Sameer Singh and Carlos Guestrin. '"Why Should I Trust You?": Explaining the Predictions of Any Classifier'. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD '16. New York, NY, USA: Association for Computing Machinery, Aug. 2016, pp. 1135–1144. ISBN: 978-1-4503-4232-2. DOI: 10.1145/2939672.2939778. URL: https://dl.acm.org/doi/10.1145/2939672.2939778 (visited on 01/06/2026).

[35] Cynthia Rudin. 'Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead'. In: *Nature Machine Intelligence* 1.5 (May 2019), pp. 206–215. ISSN: 2522-5839. DOI: 10.1038/s42256-019-0048-x. URL: https://www.nature.com/articles/s42256-019-0048-x (visited on 01/06/2026).

[36] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh and Dhruv Batra. 'Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization'. In: *2017 IEEE International Conference on Computer Vision (ICCV)*. Oct. 2017, pp. 618–626. DOI: 10.1109/ICCV.2017.74. URL: https://ieeexplore.ieee.org/document/8237336 (visited on 01/06/2026).

[37] Irhum Shafkat. *Intuitively Understanding Variational Autoencoders*. Feb. 2018. URL: https://medium.com/data-science/

intuitively - understanding - variational - autoencoders - 1bfe67eb5daf (visited on 14/05/2026).

[38] Karen Simonyan, Andrea Vedaldi and Andrew Zisserman. *Deep Inside Convolutional Networks: Visualising Image Classification Models and Saliency Maps*. Apr. 2014. DOI: 10.48550/arXiv.1312.6034. arXiv: 1312.6034 [cs.CV]. URL: http://arxiv.org/abs/1312.6034 (visited on 11/06/2026).

[39] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng and Christopher Potts. 'Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank'. In: *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Ed. by David Yarowsky, Timothy Baldwin, Anna Korhonen, Karen Livescu and Steven Bethard. Seattle, Washington, USA: Association for Computational Linguistics, Oct. 2013, pp. 1631–1642. URL: https://aclanthology.org/D13-1170/ (visited on 18/06/2026).

[40] Chung-En Sun, Tuomas Oikarinen, Berk Ustun and Tsui-Wei Weng. 'Concept Bottleneck Large Language Models'. In: *International Conference on Learning Representations* 2025 (May 2025), pp. 89371–89411. URL: https://proceedings.iclr.cc/paper_files/paper/2025/hash/de4ce91dfe56b919ee1c228d6a78f866-Abstract-Conference.html (visited on 01/06/2026).

[41] Adly Templeton et al. *Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet*. URL: https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html#appendix-citation (visited on 20/10/2025).

[42] Robert Tibshirani. 'Regression Shrinkage and Selection Via the Lasso'. In: *Journal of the Royal Statistical Society: Series B (Methodological)* 58.1 (Jan. 1996), pp. 267–288. ISSN: 0035-9246. DOI: 10.1111/j.2517-6161.1996.tb02080.x. URL: https://doi.org/10.1111/j.2517-6161.1996.tb02080.x (visited on 29/05/2026).

[43] Aaron van den Oord, Oriol Vinyals and Koray Kavukcuoglu. 'Neural Discrete Representation Learning'. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: https://papers.nips.cc/paper/2017/hash/7a98af17e63a0ac09ce2e96d03992fbc-Abstract.html (visited on 11/06/2026).

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser and Illia Polosukhin. 'Attention Is All You Need'. In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html (visited on 11/06/2026).

[45] Sandra Wachter, Brent Mittelstadt and Chris Russell. *Counterfactual Explanations Without Opening the Black Box: Automated Decisions and the GDPR*. SSRN Scholarly Paper. Rochester, NY, Oct. 2017. DOI: 10.2139/ssrn.3063289. Social Science Research Network: 3063289. URL: https://papers.ssrn.com/abstract=3063289 (visited on 11/06/2026).

[46] Benjamin Wright and Lee Sharkey. 'Addressing Feature Suppression in SAEs — AI Alignment Forum'. In: (Feb. 2024). URL: https://www.alignmentforum.org/posts/3JuSjTZyMzaSeTxKk/addressing-feature-suppression-in-saes (visited on 29/05/2026).

COLOPHON

This document was typeset using the typographical look-and-feel `classicthesis` developed by André Miede and Ivo Pletikosić. The style was inspired by Robert Bringhurst's seminal book on typography "*The Elements of Typographic Style*". `classicthesis` is available for both LaTeX and LyX:

https://bitbucket.org/amiede/classicthesis/

Happy users of `classicthesis` usually send a real postcard to the author, a collection of postcards received so far is featured here:

http://postcards.miede.de/

Thank you very much for your feedback and contribution.